Face Recognition Based Intelligent Access Control System with Attendance and Anomaly Analysis

By

Fizza Arooj Anwer
(2022-GCUF-02717)

Ghulam Fatima
(2022-GCUF-02729)

BACHELOR OF SCIENCE IN
INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY
GOVERNMENT COLLEGE UNIVERSITY, FAISALABAD.

June 2026

DECLARATION

The work reported in this documentation was carried out by us under the supervision of **Prof. Dr. Rabia Saleem** Department of Information Technology Government College University, Faisalabad, Pakistan.
We hereby declare that the title of project **"Face Recognition Based Intelligent Access Control System with Attendance and Anomaly Analysis"** and the contents of the documentation are the product of my own work and no part has been copied from any published source (except the references, standard mathematical or genetic models /equations /formulas/protocols etc.) We further declare that this work has not been submitted for award of any other degree /diploma. The University may take action if the information provided is found inaccurate at any stage.

Signature of the Student
Fizza Arooj Anwer
2022-GCUF-02717

Signature of the Student
Ghulam Fatima
2022-GCUF-02729

CERTIFICATE BY SUPERVISORY COMMITTEE

We certify that the contents and form of documentation submitted by Ms. Fizza Arooj Anwer Registration No: 2022-GCUF-02717 and by Ms. Ghulam Fatima Registration No: 2022-GCUF-02729. Has been Found satisfactory and in accordance with the prescribed format. We recommend it to be processed for the evaluation by the External Examiner for the award of Degree.

Supervisor

Signature of Supervisor …….
Name…….
Designation with stamp…….

Member of Supervisory Committee

Signature ……
Name…….....
Designation with stamp…….....

Member of Supervisory Committee

Signature ……
Name…….....
Designation with stamp…….....

Chairperson

Signature ……
Name…….....
Designation with stamp…….....

DEDICATION

To

Our Parents & Families

Whose boundless love, sleepless nights, silent prayers, and unshakeable faith in us made every difficult moment bearable and every achievement possible.

And to Our Teachers

Whose dedication to knowledge and commitment to student success shaped not only our academic journey but our character as well.

And to every student who builds something meaningful from nothing.

ACKNOWLEDGEMENTS

All praise to Almighty Allah, the Most Gracious and the Most Merciful, and His Holy Prophet Muhammad (Peace Be Upon Him), the most perfect and exalted among all born upon the surface of the earth, who is forever a torch of guidance and knowledge for humanity as a whole.

We would like to express our deepest gratitude and sincere appreciation to our project supervisor, **Prof. Dr. Rabia Saleem**, for her invaluable guidance, continuous encouragement, and patient mentorship throughout the entire duration of this project.

Our sincere thanks are extended to the Chairperson and all faculty members of the Department of Information Technology, Government College University, Faisalabad, for equipping us with the knowledge, skills, and academic foundation required for this work.

We are deeply grateful to our parents, siblings, and families for their unwavering support, moral encouragement, and sacrifices throughout our academic journey.

We also acknowledge the open-source communities behind Python, OpenCV, DeepFace, Pandas, and Matplotlib, whose freely available tools and documentation made this work possible.

Fizza Arooj Anwer
Ghulam Fatima

ABSTRACT

Access control systems constitute a fundamental layer of organisational security infrastructure. Conventional mechanisms such as PIN-based authentication, RFID swipe cards, and physical keys are susceptible to theft, duplication, sharing, and loss, rendering them inadequate for environments that require robust, reliable, and user-friendly security.

This Final Year Project presents the complete design and implementation of a Face Recognition Based Intelligent Access Control System with Attendance and Anomaly Analysis, a comprehensive, software-based biometric security solution developed entirely in Python 3.10 using freely available open-source libraries. The system authenticates individuals through real-time facial recognition powered by the DeepFace library employing the VGG-Face deep convolutional neural network model, which extracts 2,622 unique facial feature embeddings per individual.

A central and distinguishing contribution of this work is the implementation of a Challenge-Response Presentation Attack Detection (PAD) mechanism. Rather than passive liveness detection, the system presents the user with a randomly selected head movement instruction at scan time and measures the resulting face centre displacement in pixel space. Static photographs and pre-recorded video replays are incapable of dynamically responding to these randomised instructions, effectively blocking the most common spoofing attack vectors.

The system is structured into five integrated modules: (1) Face Recognition and User Registration; (2) Access Control and Event Logging; (3) Automated Attendance Management; (4) Salary Estimation; and (5) Rule-Based Anomaly Detection. All five modules are integrated into a professional dark-themed desktop dashboard built with Tkinter.

Testing was conducted across 26 test cases covering unit, integration, system, and user acceptance testing. All 12 functional requirements were verified as fully implemented and passing. The system successfully blocked all tested spoofing attempts, correctly identified registered users, accurately recorded attendance, computed salaries, and flagged all injected anomalous patterns.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

SOFTWARE REQUIREMENT SPECIFICATION

1.1 Introduction

The Face Recognition Based Intelligent Access Control System is a comprehensive, software-based biometric security platform designed to overcome the fundamental limitations of conventional credential-based access control mechanisms in institutional and organisational environments. Traditional access control relies on physical artefacts such as PIN codes, RFID cards, keys, and passwords, all of which are vulnerable to loss, theft, duplication, sharing, and social engineering.

This project addresses these shortcomings by leveraging the power of deep learning-based facial recognition combined with active liveness detection to create a system that authenticates individuals using biometric traits that cannot be shared, transferred, or replicated. The system further extends beyond authentication by automating attendance tracking, salary computation, and anomaly-based security monitoring.

This Software Requirements Specification (SRS) document formally defines the functional and non-functional requirements of the system, establishes the project scope and feasibility analysis, identifies all stakeholders, and presents the development schedule.

The system operates by capturing a live video stream from a standard USB or integrated webcam and processing each frame through a multi-stage pipeline. In the first stage, the Haar Cascade face detector from OpenCV identifies the presence and bounding coordinates of a human face within the frame. In the second stage, the challenge-response liveness module presents a random head movement instruction on screen and measures pixel-space displacement of the detected face centroid over time.

Only if the displacement threshold is met in the correct direction — corresponding to the displayed instruction — is the liveness check considered passed. In the third stage, DeepFace extracts a 2,622-dimensional embedding from the live frame using the pre-trained VGG-Face convolutional neural network and computes the cosine distance between the live embedding and every stored registration embedding, granting access if the best-match distance falls below the model's internal threshold.

The significance of this project extends beyond its immediate institutional deployment context. It demonstrates that robust biometric access control, previously achievable only through expensive dedicated hardware such as commercial fingerprint readers or iris scanners, can be realised entirely in software using freely available academic research models.

This finding has practical implications for educational institutions, small businesses, and non-governmental organisations in resource-constrained environments where commercial biometric solutions are economically inaccessible. Furthermore, the modular architecture of the system ensures that individual components — particularly the face recognition model — can be swapped or upgraded independently as the state of the art advances without requiring a complete system rebuild.

1.1.1 Background and Motivation

The increasing availability of high-quality webcams, the maturity of open-source deep learning libraries, and the reduced computational cost of neural network inference have

made software-based biometric authentication practically viable on standard consumer hardware. The DeepFace library, which wraps multiple state-of-the-art face recognition models including

VGG-Face, ArcFace, and FaceNet, enables face verification with accuracy comparable to commercial biometric systems without any hardware investment beyond an existing webcam.

The motivation for this project arose from the observation that most academic and small business environments continue to rely on manual attendance registers and PIN-based door access, both of which are inefficient and insecure.

This project demonstrates that a robust, multi-functional biometric access control system can be built entirely from open-source components, making it accessible to institutions with limited budgets.

The choice of the VGG-Face model as the recognition backbone is deliberate. Published by Parkhi et al. at the Visual Geometry Group, Oxford (2015), VGG-Face was trained on over 2.6 million facial images of 2,622 unique identities and achieves 98.95% verification accuracy on the Labeled Faces in the Wild (LFW) benchmark.

The DeepFace library wraps VGG-Face in a Python interface that abstracts the preprocessing pipeline (alignment, normalisation, and embedding extraction), exposing a single verify() function that accepts two image paths and returns a boolean verification result along with the cosine distance. This abstraction level was essential for maintaining a clean, testable module boundary in the system architecture.

From a security engineering perspective, the most significant limitation of many deployed face recognition systems is their vulnerability to Presentation Attacks (PA), defined under ISO/IEC 30107-1:2023 as the presentation of an artefact or human characteristic to the biometric capture subsystem in a way that interferes with the policy of the biometric system.

The most common presentation attacks are printed photograph attacks, in which a high-resolution photograph of the target individual is held in front of the camera, and video replay attacks, in which a pre-recorded video of the target individual is played on a screen.

The Challenge-Response PAD implemented in this project addresses both attack vectors through the same mechanism: because both attacks present a static or pre-determined visual to the camera, neither can dynamically respond to a randomly generated head movement instruction that was not known at the time the attack artefact was prepared.

Project Objectives

General Objectives

The general objective is to develop an intelligent, integrated biometric access control system capable of verifying user identity through facial recognition, automating attendance management, computing payroll, and detecting anomalous access patterns with minimal human intervention.

Specific Objectives

- To implement user registration by capturing five facial samples per employee and storing biometric templates in a structured database.
- To implement face recognition using the DeepFace VGG-Face model with 2,622 facial feature embeddings for identity verification.
- To develop a Challenge-Response Presentation Attack Detection (PAD) mechanism using head movement instructions to prevent spoofing.
- To automate employee attendance recording by logging entry and exit timestamps and computing working hours upon successful face verification.
- To calculate employee salary estimates from attendance hours and configurable per-employee hourly rates.
- To implement a five-rule anomaly detection engine that identifies and classifies suspicious access patterns by risk level.
- To integrate all five modules into a professional Tkinter-based desktop dashboard with a live camera feed and real-time status indicators.
- To generate automated visual reports including bar and pie charts for attendance, salary, and anomaly analysis.

1.2 Scope of the System

The scope of this project encompasses the design, development, testing, and documentation of a single-site, software-based biometric access control system for deployment on a Windows desktop computer equipped with a standard webcam.

1.2.1 In-Scope Features

- Biometric user registration and deletion via webcam.
- Challenge-response active liveness detection for anti-spoofing.
- Real-time facial recognition using the VGG-Face deep learning model.
- Access grant/deny decision making with event logging.
- Security alert generation with intruder photograph capture.
- Automated entry and exit attendance recording.
- Working hours computation and attendance report generation.
- Employee salary estimation with payroll report generation.
- Five-rule anomaly detection with risk classification.
- Professional GUI dashboard with live camera feed and chart visualization.

1.2.2 Out-of-Scope Features

- Network-based multi-door or multi-site access control.
- Physical lock hardware integration via GPIO or relay control.
- Cloud-based data synchronisation or remote monitoring.
- Mobile application interface or web-based dashboard.
- Active Directory or HR system integration.

1.3 Feasibility Analysis

1.3.1 Technical Feasibility

The system utilises Python 3.10 as its primary development language, which provides mature, stable support for all required libraries. The DeepFace library provides access to the pre-trained VGG-Face model for facial embedding extraction, enabling accurate face verification without GPU acceleration — standard consumer CPU hardware is sufficient for real-time inference at the frame-sampling rate used by the system. All chosen technologies are proven in production environments, and all dependencies are freely available under permissive open-source licences.

A key technical risk during development was the incompatibility between the OpenCV imshow() function and the Tkinter event loop on Windows. Both subsystems require control of the main GUI thread, creating a concurrency conflict when both are used simultaneously in a single-threaded application.

This was resolved through a carefully engineered threading model: liveness detection uses a root.after() state machine that runs one OpenCV frame per callback invocation on the main thread, while face recognition runs in a daemon thread that dispatches results back to the main thread using root.after(0, lambda) closures. This threading model was validated across Windows 10 and Windows 11 and is documented in detail in Chapter 4.

A secondary technical risk was the thread-unsafe behaviour of Matplotlib's interactive backends (TkAgg, Qt5Agg) when invoked from background threads. This was resolved by configuring all chart-generating modules to use the Agg (non-interactive, file-output-only) backend at import time.

Charts are rendered to PNG files in the reports/ directory and subsequently loaded into Tkinter Toplevel windows using PIL.Image and ImageTk.PhotoImage, entirely bypassing the interactive Matplotlib display system. This approach is thread-safe, produces publication-quality PNG output at configurable DPI, and does not require any additional graphical framework beyond Tkinter.

1.3.2 Economic Feasibility

The system requires no hardware investment beyond a standard desktop or laptop computer already available at the deployment site. No commercial software licences are required. Deployment cost is therefore limited to the time required for software installation and user registration, making the system highly economically viable for institutional adoption.

A comparative cost analysis further reinforces this conclusion. Commercial biometric access control systems typically cost between USD 300 and USD 2,000 per door controller unit, excluding installation, cabling, and recurring maintenance contracts. Enterprise face recognition APIs such as Amazon Rekognition and Microsoft Azure Face charge per-API-call fees that accumulate to significant monthly costs at scale.

In contrast, the total cost of deploying this system consists exclusively of the price of a standard webcam (approximately PKR 3,000 to 8,000) and the time required for initial setup, which UAT demonstrated can be completed within one hour by a non-technical user. For an institution with 10 access points, this represents a saving of potentially hundreds of thousands of rupees relative to commercial alternatives, with no ongoing licensing costs.

1.3.3 Operational Feasibility

The system is designed for use by non-technical administrative staff. The GUI provides clearly labelled controls for all operations. The registration process requires only entering a name and pressing a key five times in front of the webcam. A non-technical user can be fully operational within one hour of installation.

The operational workflow was designed to minimise cognitive load on the end user. For an entering employee, the complete interaction consists of three steps:

(1) approach the camera station

(2) follow the displayed head movement instruction

(3) wait for the AUTHORIZED confirmation message.

No PIN to remember, no card to carry, and no button to press. The system handles all decision-making, logging, and attendance recording automatically following a successful scan. Exit scanning follows the identical procedure and additionally displays the computed hours worked in a popup notification, providing the employee with immediate feedback.

From an organisational change management perspective, the system requires minimal retraining. The most significant change to existing workflows is the replacement of the physical attendance register with biometric scanning.

This transition eliminates proxy attendance — the practice of one employee signing in for an absent colleague — which is a significant source of payroll fraud in manual attendance systems. The security officer role benefits most significantly from the anomaly detection module, which replaces the time-consuming manual review of access logs with an automated, rule-based classification system that surfaces only flagged records for human review.

1.4 Stakeholder

- System Administrator: Responsible for user registration and deletion, system configuration, and report generation.
- Registered Employees: Individuals enrolled in the system who authenticate via face scan for entry and exit.
- Human Resources / Management: Review attendance reports, salary estimates, and anomaly detection results.
- Security Officer: Reviews security alerts, failed authentication logs, and captured intruder photographs.
- IT Support Personnel: Responsible for software maintenance, dependency updates, and database backups.
- Academic Supervisory Committee: Evaluates the system against FYP requirements.

1.5 Users of the System

1.5.1 Administrator

The administrator has unrestricted access to all system functions through the GUI sidebar. Key responsibilities include registering new employees, managing the user database, initiating manual report generation, reviewing anomaly detection results, and acknowledging security alerts.

1.5.2 Registered Employee

The registered employee interacts with the system solely through biometric face scanning during working hours. No manual input is required during normal entry or exit operations.

6 Functional Requirement

1.6.1 List of Functional Requirements

ID	Name	Description	Priority
FR-01	User Registration	Register a new user by capturing 5 facial samples via webcam and storing templates in the database	High
FR-02	Liveness Detection	Present 2 random challenge-response head movement instructions and verify completion before recognition	High
FR-03	Face Recognition	Compare live frames against registered templates using DeepFace VGG-Face and return RECOGNIZED or NOT RECOGNIZED	High
FR-04	Access Control	Grant or deny access based on recognition result; log each event with timestamp, name, and status	High
FR-05	Security Alert	Trigger security alert popup and capture intruder photo after 3 consecutive failed attempts	High
FR-06	Entry Recording	Record employee entry time in attendance.csv upon successful recognition via Entry scan	High
FR-07	Exit Recording	Record exit time and compute hours worked upon successful recognition via Exit scan	High
FR-	Attendance	Generate summary of days present, total and average hours	Medium

08	Report	per employee with charts	
FR-09	Salary Estimation	Calculate salary as total hours x hourly rate and produce payroll report with charts	Medium
FR-10	Anomaly Detection	Apply 5 detection rules to logs and attendance data; classify results by risk level	Medium
FR-11	Chart Generation	Generate bar and pie charts for attendance, salary, and anomaly data as PNG popup windows	Medium
FR-12	User Deletion	Remove a registered user and all associated face data and records from the system	Medium

1.5.3 Requirements in Requirement Shell Format

Table 1: Requirement Shell - User Registration (Req-01)

Requirement #:	Req-01	**Type:**	Functional
Name:	User Registration		
Description:	The system shall enable the administrator to register a new employee by capturing five facial image samples using the connected webcam. The face recognition model processes each sample and stores the user's name and face template path in the users.csv database.		
Rationale:	To ensure that only pre-approved, registered individuals can be authenticated by the biometric system.		
Originator:	System Administrator		
Fit Criterion:	The registered user record appears in users.csv with a valid face_path. All five sample images are saved to database/faces/. A success confirmation is displayed in the GUI.		
Cust. Satisfaction:	4	**Cust. Dissatisfaction:**	9
Priority:	High	**Conflicts:**	Nil
Supporting Materials:	N/A		
History:	Initial definition - June 2026		

Table 2: Requirement Shell - Liveness Detection (Req-02)

Requirement #:	Req-02	**Type:**	Functional
Name:	Challenge-Response Liveness Detection		
Description:	The system shall randomly select two head movement challenges from the available set and display them as on-screen instructions. The system shall measure face centre pixel displacement per frame against a threshold of 30 pixels. Both challenges must be passed within 10 seconds each before proceeding to face recognition.		
Rationale:	To prevent Presentation Attacks using static photographs or pre-recorded video replays, which cannot dynamically respond to randomised instructions.		
Originator:	System (automated, triggered at scan initiation)		
Fit Criterion:	A static photograph cannot pass the liveness check. A real person who follows the instructions passes within the time window.		
Cust. Satisfaction:	5	**Cust. Dissatisfaction:**	10
Priority:	High	**Conflicts:**	Nil
Supporting Materials:	N/A		
History:	Initial definition - June 2026		

Table 3: Requirement Shell - Face Recognition (Req-03)

Requirement #:	Req-03	**Type:**	Functional
Name:	Face Recognition and Verification		
Description:	The system shall compare a live webcam frame against the stored VGG-Face template of each registered user using DeepFace.verify(). The comparison shall return RECOGNIZED with the matched user's name, or NOT RECOGNIZED if no match is found within the 10-second timeout.		
Rationale:	Core authentication mechanism that determines whether the presenting individual is a registered authorised user.		
Originator:	System (automated, following liveness check)		
Fit Criterion:	Registered users are correctly identified. Unregistered individuals are denied. System auto-exits 1.5 seconds after a successful match.		
Cust. Satisfaction:	5	**Cust. Dissatisfaction:**	9
Priority:	High	**Conflicts:**	Nil
Supporting Materials:	N/A		
History:	Initial definition - June 2026		

Table 4: Requirement Shell - Access Control (Req-04)

Requirement #:	Req-04	**Type:**	Functional
Name:	Access Control and Event Logging		
Description:	The system shall process the recognition result and log an access event to access_logs.csv with the fields: date, time, name, status (GRANTED/DENIED/SECURITY_ALERT), and time_of_day classification. The failed attempts counter shall be incremented on each denial.		
Rationale:	To maintain a complete, auditable record of all access events for security review and anomaly detection.		
Originator:	System (automated, triggered after recognition)		
Fit Criterion:	Every access attempt is logged. GRANTED events include the authenticated user's name. DENIED events include attempt count.		
Cust. Satisfaction:	4	**Cust. Dissatisfaction:**	8
Priority:	High	**Conflicts:**	Nil
Supporting Materials:	N/A		
History:	Initial definition - June 2026		

Table 5: Requirement Shell - Attendance Recording (Req-05)

Requirement #:	Req-05	**Type:**	Functional
Name:	Automated Attendance Recording		
Description:	The system shall record employee entry time to attendance.csv upon successful face scan via the Entry button. Upon Exit scan, the system shall locate the employee's open entry record for the current date, update the exit time, and compute hours_worked.		
Rationale:	To replace manual attendance registers with an automated, tamper-resistant biometric record.		
Originator:	System (automated, triggered after successful authentication)		
Fit Criterion:	Each employee entry creates a new attendance row. Each exit updates the corresponding row with exit time and computed hours.		
Cust. Satisfaction:	4	**Cust. Dissatisfaction:**	8
Priority:	High	**Conflicts:**	Nil
Supporting Materials:	N/A		
History:	Initial definition - June 2026		

Table 6: Requirement Shell - Salary Estimation (Req-06)

Requirement #:	Req-06	**Type:**	Functional
Name:	Salary Estimation and Reporting		
Description:	The system shall read total hours worked per employee from the attendance CSV, multiply by the employee's configured hourly rate (default: Rs. 150/hr), and produce a payroll report. Bar and pie chart visualisations shall be generated and displayed in the GUI.		
Rationale:	To automate payroll computation from biometric attendance data, reducing manual calculation errors.		
Originator:	System Administrator, HR/Management		
Fit Criterion:	Salary report accurately reflects total_hours x hourly_rate per employee. Charts render correctly.		
Cust. Satisfaction:	3	**Cust. Dissatisfaction:**	7
Priority:	Medium	**Conflicts:**	Nil
Supporting Materials:	N/A		
History:	Initial definition - June 2026		

Table 7: Requirement Shell - Anomaly Detection (Req-07)

Requirement #:	Req-07	**Type:**	Functional
Name:	Rule-Based Anomaly Detection		
Description:	The system shall apply five detection rules to access_logs.csv and attendance.csv to identify suspicious patterns. Each detected anomaly shall be classified by risk level (HIGH/MEDIUM/LOW) and saved to reports/anomaly_report.csv.		
Rationale:	To provide automated, continuous security monitoring without requiring manual log review.		
Originator:	System Administrator, Security Officer		
Fit Criterion:	All five rules correctly flag expected records when tested with injected anomalous data. Risk classification matches expected levels.		
Cust. Satisfaction:	4	**Cust. Dissatisfaction:**	7
Priority:	Medium	**Conflicts:**	Nil
Supporting Materials:	N/A		
History:	Initial definition - June 2026		

1.5.4 Non-Functional Requirements

Performance

- The complete liveness check and face recognition cycle shall complete within 15 seconds under normal operating conditions.
- The live camera feed shall maintain a minimum frame rate of 25 fps during background operation.
- Attendance and anomaly report generation shall complete within 3 seconds for datasets up to 1,000 records.

Security

- All access events (GRANTED, DENIED, SECURITY_ALERT) shall be logged immutably with timestamps.
- Intruder photographs shall be automatically captured on security alert trigger and stored to disk.
- The liveness check shall prevent static photograph and video replay attacks in all tested scenarios.

Reliability

- The webcam feed shall automatically recover after each scan without requiring manual restart.
- All CSV write operations shall be atomic to prevent partial record corruption.

Usability

- On-screen liveness instructions shall be displayed in large, high-contrast text visible from 60 cm.
- A non-technical user shall be able to complete a registration and entry scan within 5 minutes of first use.

Portability

- The system shall operate on Windows 10 (64-bit) and Windows 11 without modification.
- All dependencies shall be installable via pip from PyPI without requiring administrator privileges.

1.7 Schedule of the Project

The project followed an Agile iterative development methodology with seven defined sprints.

Table 8: Project Schedule - Sprint Timeline

Sprint	Phase	Key Deliverables	Duration	Status
1	Requirements & Planning	SRS document, feasibility analysis, tool selection, environment setup	Weeks 1-3	Complete
2	Module 1: Face Recognition	User registration, DeepFace integration, initial liveness prototype	Weeks 4-7	Complete
3	Liveness Detection	Challenge-response PAD, threshold calibration, camera thread fixes	Weeks 8-9	Complete
4	Module 2: Access Control	Access logging, failed attempt counter, security alert system	Weeks 10-11	Complete
5	Modules 3 & 4	Attendance recording, hours calculation, salary computation, charts	Weeks 12-14	Complete
6	Module 5 & GUI	Anomaly detection engine, full GUI integration, chart popup windows	Weeks 15-18	Complete
7	Testing & Documentation	Unit/integration/system/UAT testing, FYP documentation	Weeks 19-22	Complete

Figure 13: Agile Sprint Gantt Chart

CHAPTER 2

ANALYSIS

2.1 Use Case Model

The use case model provides a behavioural view of the system from the perspective of its actors. Two primary actors have been identified: the Administrator, who has full access to all system management functions; and the Registered Employee, who interacts with the system exclusively through biometric scanning.

The use case model was developed using the Unified Modelling Language (UML) notation as defined in the UML 2.5.1 specification published by the Object Management Group (OMG). Use cases are expressed at the User Goal level unless otherwise specified, meaning each use case represents a goal that a specific actor wishes to achieve in a single session with the system.

The system boundary, represented as a named rectangle enclosing all use cases in the diagram, defines the scope of the software system as implemented and excludes physical infrastructure such as door locks, network components, and cloud services.

The six primary use cases were derived directly from the functional requirements defined in Chapter 1. Each functional requirement maps to at least one use case, ensuring complete traceability between the requirements specification and the behavioural model.

The Liveness Check use case is not a standalone user goal but rather a mandatory sub-function included by both Scan Entry (UC02) and Scan Exit (UC03), represented using the UML include relationship. This inclusion ensures that liveness verification is enforced by the system design rather than by implementation convention alone, making it architecturally impossible to invoke the face recognition module without first completing the liveness check.

Figure 2: Use Case Diagram
Face Recognition Based Intelligent Access Control System

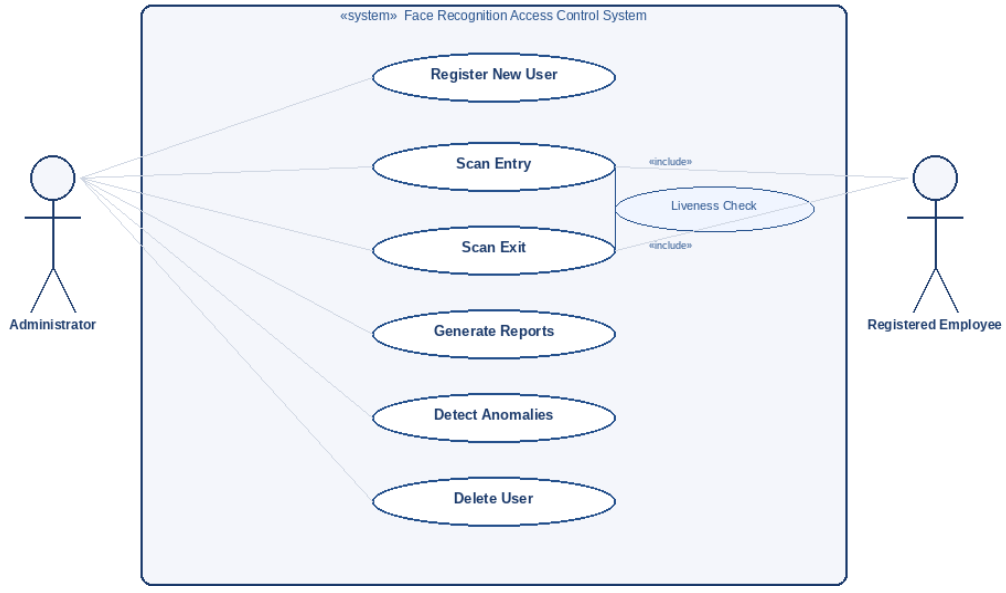


Figure 2: Use Case Diagram

2.1.1 Use Cases in Fully Dressed Format

Table 9: Fully Dressed Use Case - UC01: Register New User

UC ID	UC01	**Ref:**	SRS-UC01
UC Name	Register New User		
Level	Detailed (User Goal)		
Description	The administrator registers a new employee by entering their name and capturing five facial samples via the live webcam feed.		
Actor(s)	Administrator		
Stakeholders	Administrator, HR Management, Security Officer		
Preconditions	GUI application running. Webcam connected. No existing user with the same name.		
Success Guarantee	New user record in users.csv. Five facial samples saved. User appears in Registered Users list.		
Main Success Scenario	**Step**	**Action / System Response**	
	1	Administrator clicks Register New User. System opens registration dialog.	
	2	Administrator enters employee name and clicks Start Registration. System opens webcam window.	
	3	Administrator positions employee in front of webcam. System detects face and draws green bounding box.	
	4	Administrator presses S key. System captures one facial sample.	
	5	Administrator repeats step 4 four more times. System saves samples _s1 through _s5.	
	6	System saves user record to users.csv. Success popup displayed.	
Extensions / Alternatives	3a. No face detected: warning displayed. 2b. Cancel clicked: registration aborted.		
Frequency	Low - once per employee at onboarding		
Special Requirements	Webcam must deliver minimum 640x480 resolution.		

Table 10: Fully Dressed Use Case - UC02: Scan Entry

UC ID	UC02	**Ref:**	SRS-UC02
UC Name	Scan Entry		
Level	Detailed (User Goal)		
Description	A registered employee scans their face to record entry into the premises.		
Actor(s)	Registered Employee		

Stakeholders	Administrator, HR Management, Security Officer	
Preconditions	System running. Employee registered. Webcam connected. No open entry record for current date.	
Success Guarantee	GRANTED access event logged. Entry record created in attendance.csv. GUI updated.	
Main Success Scenario	**Step**	**Action / System Response**
	1	Click Scan Entry. System initiates liveness check.
	2	System displays first random challenge instruction with 10-second timer.
	3	Employee follows instruction. System detects face displacement >30px. Challenge 1 PASSED.
	4	System displays second random challenge. Employee passes. Challenge 2 PASSED.
	5	System opens recognition window. Runs DeepFace.verify() every 30 frames.
	6	Match found. System displays AUTHORIZED: [Name]. Waits 1.5 seconds. Closes window.
	7	System logs GRANTED event and records entry time. GUI shows Access Granted popup.
Extensions / Alternatives	3a. 10-second timeout: challenge FAILED; access denied. 5c. 3 consecutive failures: SECURITY_ALERT triggered.	
Frequency	High - multiple times daily	
Special Requirements	System must complete entry scan cycle within 15 seconds.	

Table 11: Fully Dressed Use Case - UC03: Scan Exit

UC ID	UC03	**Ref:**	SRS-UC03
UC Name	Scan Exit		
Level	Detailed (User Goal)		
Description	A registered employee scans their face to record exit. System records exit time and computes hours worked.		
Actor(s)	Registered Employee		
Stakeholders	Administrator, HR Management		
Preconditions	System running. Employee registered. Open entry record exists for current date.		
Success Guarantee	Attendance record updated with exit time and computed hours_worked.		
Main Success Scenario	**Step**	**Action / System Response**	
	1	Click Scan Exit. System initiates liveness check.	
	2	Liveness check completes (same flow as UC02).	

	3	Face recognition completes. Employee RECOGNIZED.
	4	System records exit time; computes hours = (exit - entry) in decimal hours.
	5	System displays Exit Recorded popup with hours worked.
Extensions / Alternatives	3a. Liveness fails or not recognised: denied, no exit recorded. 4a. No open entry found: warning displayed.	
Frequency	High - once daily per employee	
Special Requirements	Hours computation accurate to within 1-minute tolerance.	

Table 12: Fully Dressed Use Case - UC04: Detect Anomalies

UC ID	UC04	**Ref:**	SRS-UC04
UC Name	Detect Anomalies		
Level	Detailed (Subfunction)		
Description	Administrator initiates anomaly detection. System applies five rules, produces risk-classified report, displays results in GUI.		
Actor(s)	Administrator		
Stakeholders	Security Officer, HR Management		
Preconditions	System running. At least one data file contains records.		
Success Guarantee	Anomaly report saved to reports/. GUI table displays all anomalies. Chart popup shown.		
Main Success Scenario	**Step**	**Action / System Response**	
	1	Administrator clicks Detect Anomalies. System loads CSV files.	
	2	Rule 1: flag records with 3+ DENIED events per (name, date) as HIGH risk.	
	3	Rule 2: flag GRANTED events between 22:00 and 06:00 as MEDIUM risk.	
	4	Rule 3: flag attendance records with hours_worked < 2.0 as LOW risk.	
	5	Rule 4: flag attendance records with hours_worked > 14.0 as MEDIUM risk.	
	6	Rule 5: flag consecutive GRANTED events for same user within 5 minutes as HIGH risk.	
	7	All anomalies saved to anomaly_report.csv. GUI displays table and chart.	
Extensions / Alternatives	1a. No data files: system displays warning; detection aborted.		
Frequency	Low - periodic review daily or weekly		
Special Requirements	Detection must process up to 10,000 records in under 5 seconds.		

Table 13: Fully Dressed Use Case - UC05: Generate Reports

UC ID	UC05	**Ref:**	SRS-UC05
UC Name	Generate Reports		
Level	Detailed (Subfunction)		
Description	Administrator generates attendance, salary, or anomaly reports on demand.		
Actor(s)	Administrator		
Stakeholders	HR Management, Security Officer		
Preconditions	System running. Relevant data CSV files exist with records.		
Success Guarantee	Report CSV saved to reports/. Chart displayed in Tkinter popup window.		
Main Success Scenario	**Step**	**Action / System Response**	
	1	Administrator clicks View Attendance Report / Generate Salary / Detect Anomalies.	
	2	System computes summary using Pandas groupby aggregation.	
	3	System saves report CSV to reports/ folder.	
	4	System generates chart PNG using Matplotlib Agg backend.	
	5	Administrator views chart popup and closes with Close button.	
Extensions / Alternatives	1a. No data available: system displays warning dialog.		
Frequency	Medium - as required by administrator		
Special Requirements	Charts must render within 3 seconds.		

Table 14: Fully Dressed Use Case - UC06: Delete User

UC ID	UC06	**Ref:**	SRS-UC06
UC Name	Delete User		
Level	Detailed (Subfunction)		
Description	Administrator removes a registered user, permanently deleting all associated facial templates and database record.		
Actor(s)	Administrator		
Stakeholders	HR Management		
Preconditions	System running. Target user exists in users.csv.		
Success Guarantee	User record removed from users.csv. All facial sample images deleted. User no longer in list.		

Main Success Scenario	Step	Action / System Response
	1	Administrator clicks Delete User. System opens deletion dialog with user list.
	2	Administrator selects user. System requests confirmation.
	3	Administrator confirms. System deletes face files and removes CSV row.
	4	System displays success notification. Activity log updated.
Extensions / Alternatives	2a. Administrator cancels: no action taken. 3a. Face files already missing: system proceeds with CSV removal.	
Frequency	Low - infrequent administrative operation	
Special Requirements	Deletion must be permanent and irreversible without re-registration.	

2.2 System Sequence Diagrams

System Sequence Diagrams illustrate the exchange of messages between external actors and the system during execution of each primary use case.

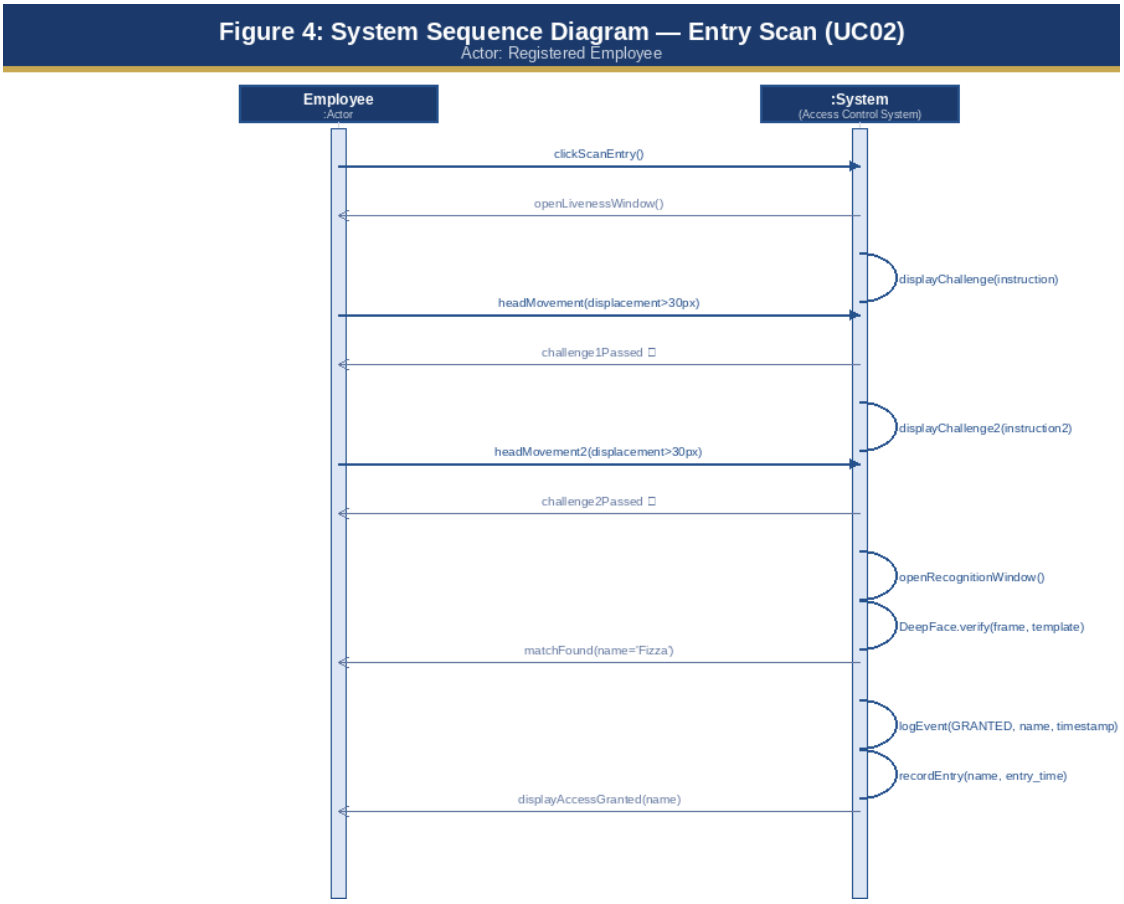


Figure 4: SSD - Entry Scan (UC02)

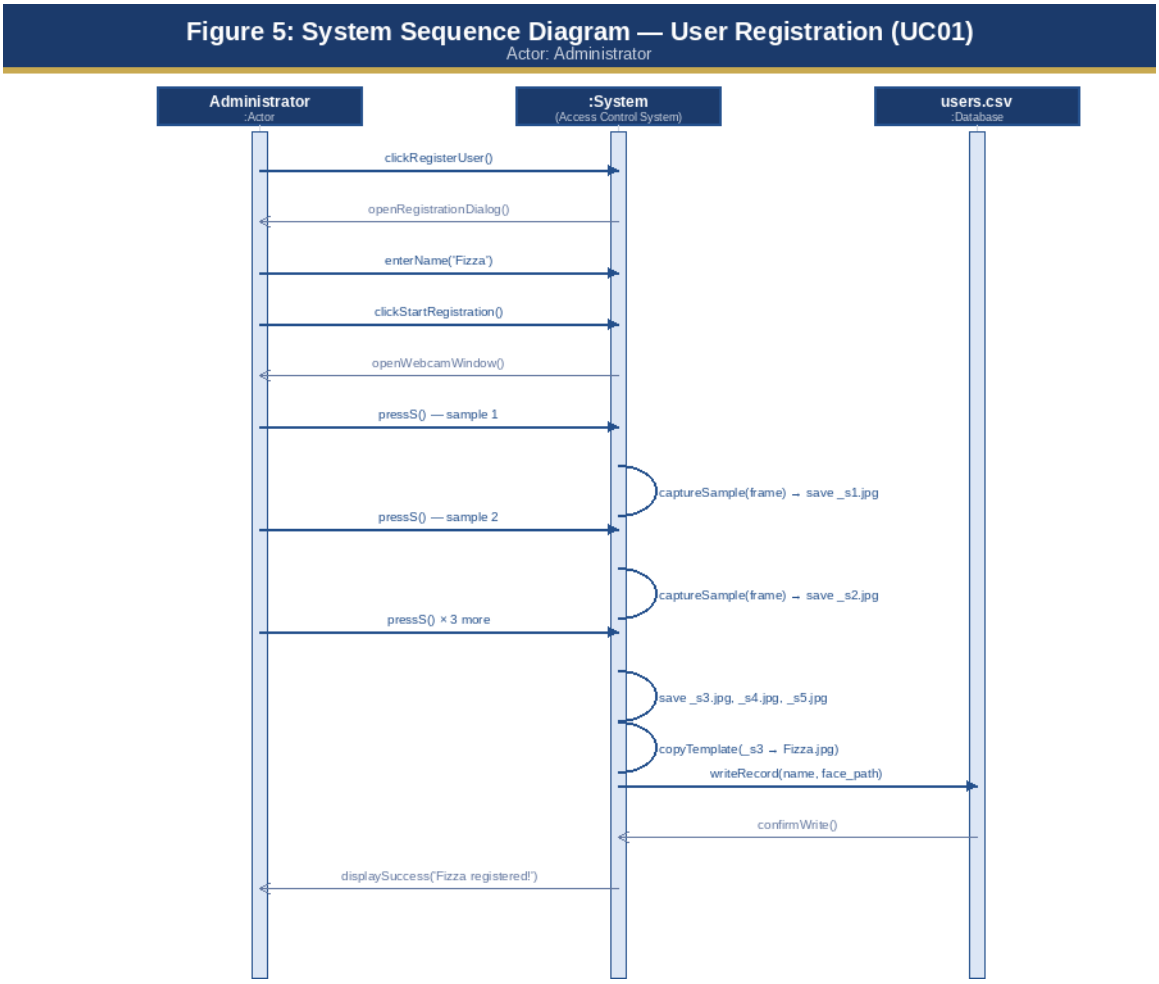


Figure 5: SSD - User Registration (UC01)

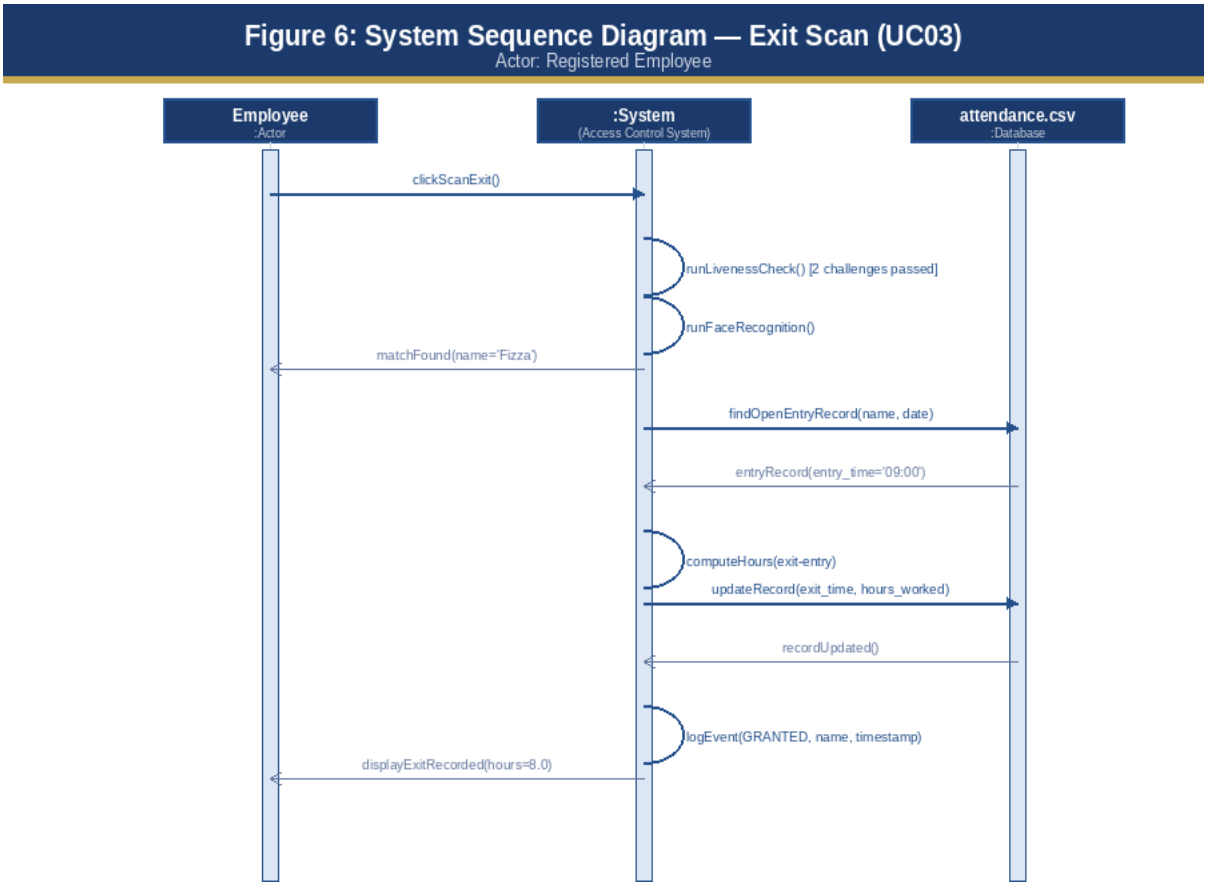


Figure 6: SSD - Exit Scan (UC03)

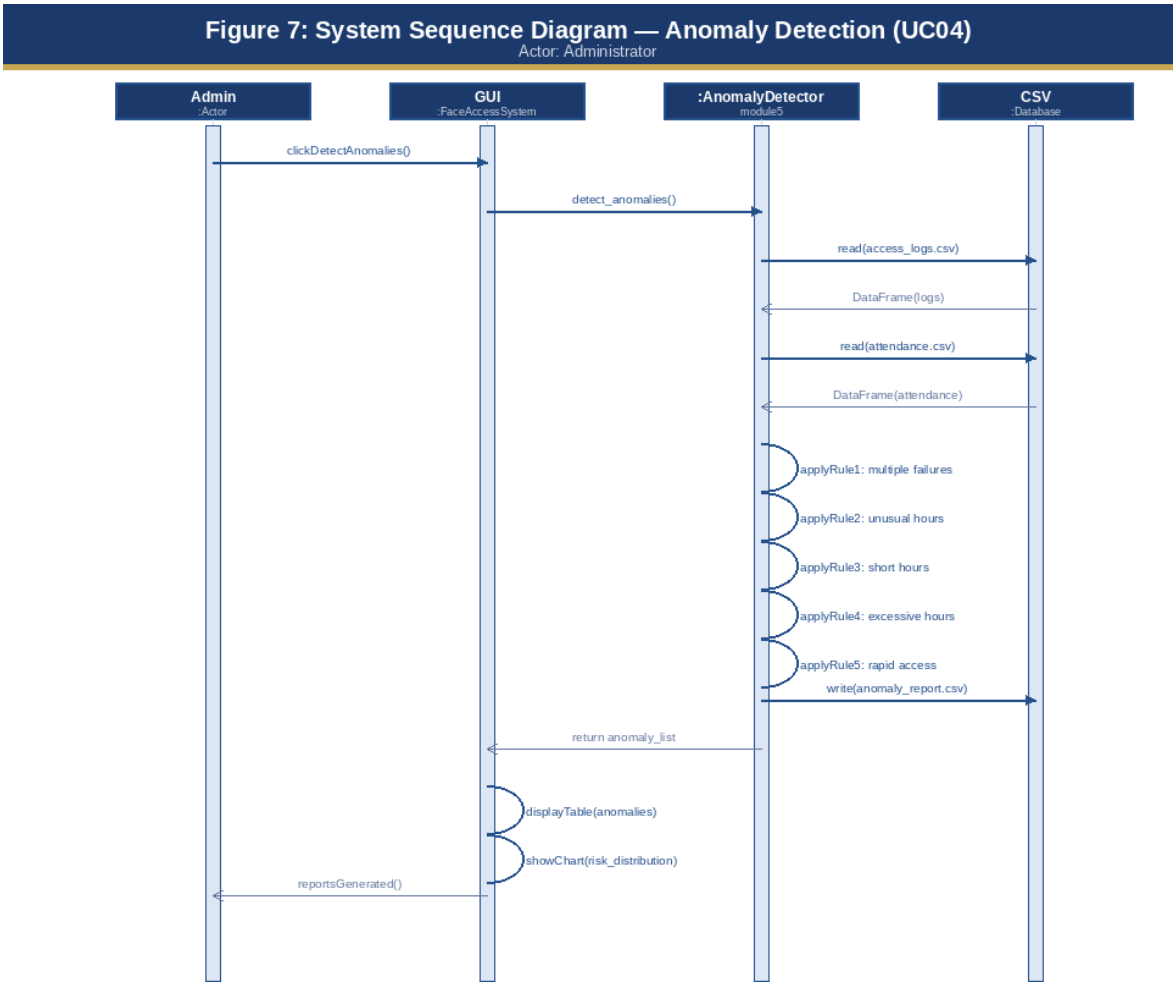


Figure 7: SSD - Anomaly Detection (UC04)

2.3 Domain Model

The domain model identifies the key conceptual classes in the problem domain and the associations between them.

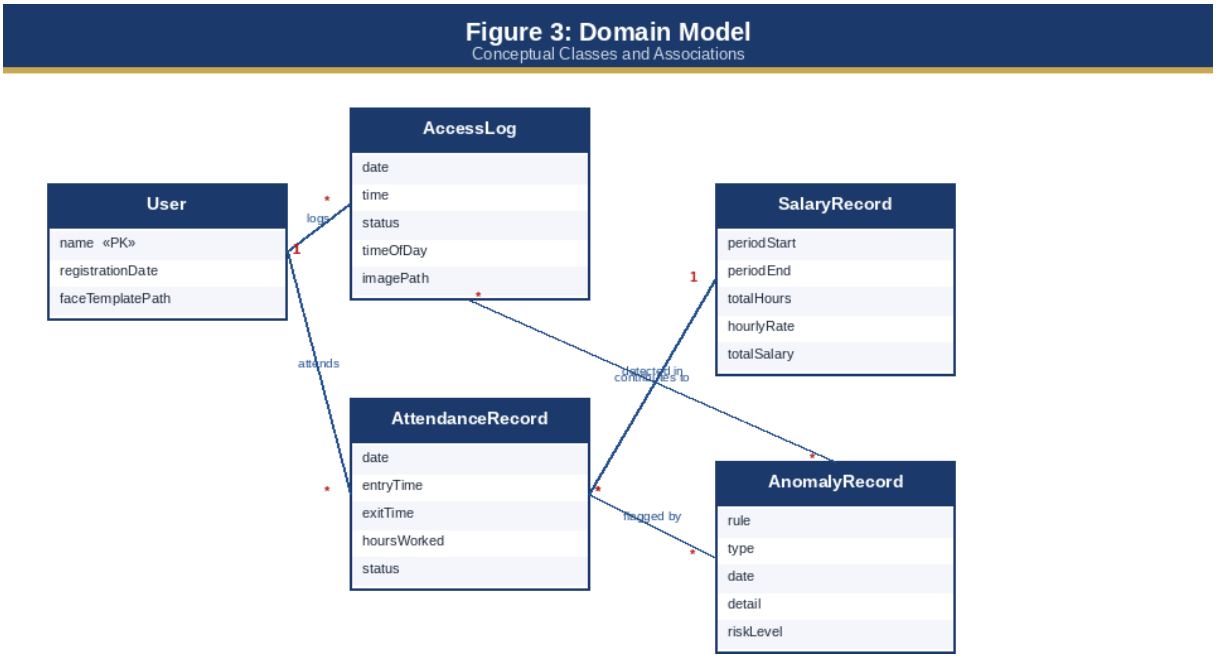


Figure 3: Domain Model - Conceptual Classes

Key Domain Classes

- User - Represents a registered employee. Primary identifier: name. Each user has many access log entries and many attendance records.
- AccessLog - Records each authentication attempt with date, time, status (GRANTED/DENIED/SECURITY_ALERT), and time_of_day classification. Source for Rules 1, 2, and 5 of the anomaly detector.
- AttendanceRecord - Tracks daily presence with entry_time, exit_time, and computed hours_worked. Source for Rules 3 and 4 of the anomaly detector and the primary input to the SalaryEngine.
- SalaryRecord - Derived from attendance data. Computed periodically by the salary module from total_hours x hourly_rate per employee.
- AnomalyRecord - Generated by the anomaly detection engine. Each record stores the triggering rule, anomaly type, affected employee, date, detail description, and risk_level (HIGH/MEDIUM/LOW).

2.4 Relationships Between Domain Classes

The associations between domain classes reflect the information flows of the system at runtime. The User class is the central entity from which all other classes derive their foreign-key relationship. A single User instance may be associated with many AccessLog records over the system lifetime, as each authentication attempt produces a log entry regardless of outcome. Similarly, one User instance may have many AttendanceRecord instances, one per working day.

The relationship between AttendanceRecord and SalaryRecord is many-to-one: many individual daily attendance records are aggregated into a single periodic salary calculation for each employee. This aggregation is performed by the SalaryEngine using Pandas groupby operations on the attendance CSV, summing hours_worked across all records within the reporting period before applying the hourly rate.

Both AccessLog and AttendanceRecord feed into the AnomalyRecord entity via the anomaly detection rules. Rules 1, 2, and 5 reference AccessLog data exclusively; Rules 3 and 4 reference AttendanceRecord data exclusively.

A single AnomalyRecord instance is generated for each individual anomalous event detected — meaning that if a single employee triggers Rule 1 (multiple failures) on three separate dates, three distinct AnomalyRecord instances are created, each with its own date and detail description. This design enables granular per-incident reporting rather than aggregated per-employee summary flags.

CHAPTER 3

PROJECT DESIGN

3.1 Introduction

This chapter presents the architectural and detailed design of the system. The design is expressed through six artefacts: the System Architecture Diagram, the Design Class Diagram, the Entity Relationship Diagram, three Sequence Diagrams, UI/UX wireframe description, and the database design.

The design process followed a top-down decomposition approach. The high-level system architecture was defined first, establishing the three-tier layered structure and the communication protocols between layers. The design class diagram was then derived from the analysis-level domain model by adding implementation-specific attributes, methods, visibility modifiers, and dependency relationships.

The ERD was designed in parallel with the class diagram to define the persistent data schema, ensuring that the data layer design directly supports the queries required by each module. The three sequence diagrams were produced last, tracing the runtime execution paths of the three most complex use cases to validate that the class and data designs were sufficient to support the required behaviour.

A deliberate design decision throughout this chapter is the separation of concerns between the five functional modules. Each module is assigned exclusive ownership of a specific data domain:
module1_iris.py owns the face template data, module2_access.py owns the access log, module3_attendance.py owns the attendance record, module4_salary.py owns salary computation, and module5_anomaly.py reads from both the access log and attendance record as read-only inputs. This ownership model prevents cross-module data corruption and makes the module boundaries independently testable, as demonstrated in Chapter 5.

3.2 System Architecture Diagram

The system is structured as a three-tier layered architecture with a clear separation of concerns between presentation, logic, and data layers.

Presentation Layer - gui.py

The GUI layer is implemented in gui.py using Tkinter. It provides the live camera feed, navigation buttons, status cards, activity log, and chart popup windows.

Logic Layer - Five Python Modules

Five independent modules: module1_iris.py (face recognition and liveness), module2_access.py (access control), module3_attendance.py (attendance), module4_salary.py (salary), and module5_anomaly.py (anomaly detection).

Data Layer - CSV File Storage

All persistent data stored in CSV files: users.csv, access_logs.csv, attendance.csv, and reports/ directory. Face templates stored as JPEG in database/faces/. The CSV format was selected over a relational database engine (such as SQLite) for three reasons:

(1) zero-configuration deployment — no database server process needs to be started or managed.

(2) human-readable format — system administrators can inspect, backup, and audit records using any text editor or spreadsheet application

(3) portability — the entire database can be copied with a single file copy operation.

3.2.1 Layer Communication Protocols

The presentation layer communicates with the logic layer exclusively through direct Python function calls. All module functions are designed to be stateless — they accept parameters, perform their computation, write to the appropriate CSV file if required, and return a result. No shared global state exists between modules, and no module directly imports from another module except through the GUI layer.

This design enforces unidirectional dependencies: gui.py depends on all five modules, but no module depends on any other module, making each module independently testable in isolation.

The logic layer communicates with the data layer through Python's built-in csv module for write operations and the Pandas library for read and aggregation operations. Write operations use csv.DictWriter with explicit fieldnames lists to ensure consistent column ordering across all platforms.

Read operations use pd.read_csv() with explicit dtype specifications where column types are critical (for example, hours_worked is coerced to numeric using pd.to_numeric() with errors='coerce' to handle empty exit_time cells gracefully). All file paths are defined as module-level constants to ensure a single point of change for deployment path configuration.

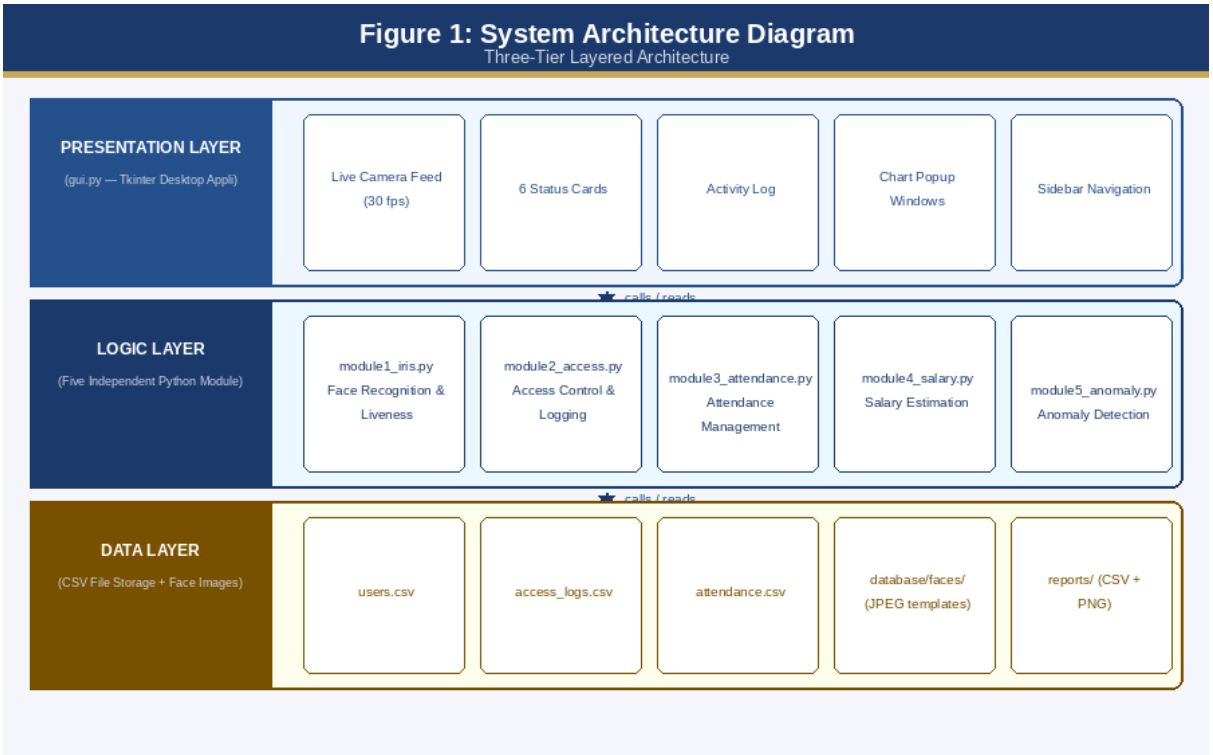


Figure 1: System Architecture - Three-Tier Layered Diagram

3.3 Design Class Diagram

FaceRecognition (module1_iris.py)

- Attributes: face_detector, DATABASE_PATH, FACES_PATH, USERS_FILE
- Methods: register_user(), recognize_user(), check_liveness(), run_challenge(), compare_faces(), save_user(), load_users(), delete_user()

AccessControl (module2_access.py)

- Attributes: LOGS_FILE, ACCESS_STATES
- Methods: process_access(), log_event(), save_alert_image(), get_statistics()

AttendanceManager (module3_attendance.py)

- Attributes: ATTENDANCE_FILE, REPORTS_PATH
- Methods: record_entry(), record_exit(), generate_report(), plot_attendance(), todays_summary()

SalaryEngine (module4_salary.py)

- Attributes: SALARY_FILE, DEFAULT_RATE, HOURLY_RATES
- Methods: generate_salary_report(), calculate_salary(), plot_salary(), get_rate(), set_hourly_rate()

AnomalyDetector (module5_anomaly.py)

- Attributes: ANOMALY_FILE, MAX_FAILED_PER_DAY, UNUSUAL_HOUR_START, MIN_WORK_HOURS, MAX_WORK_HOURS, RAPID_ACCESS_MINS
- Methods: detect_anomalies(), check_failed_attempts(), check_unusual_hours(), check_working_hours(), check_rapid_access(), plot_anomaly_chart()

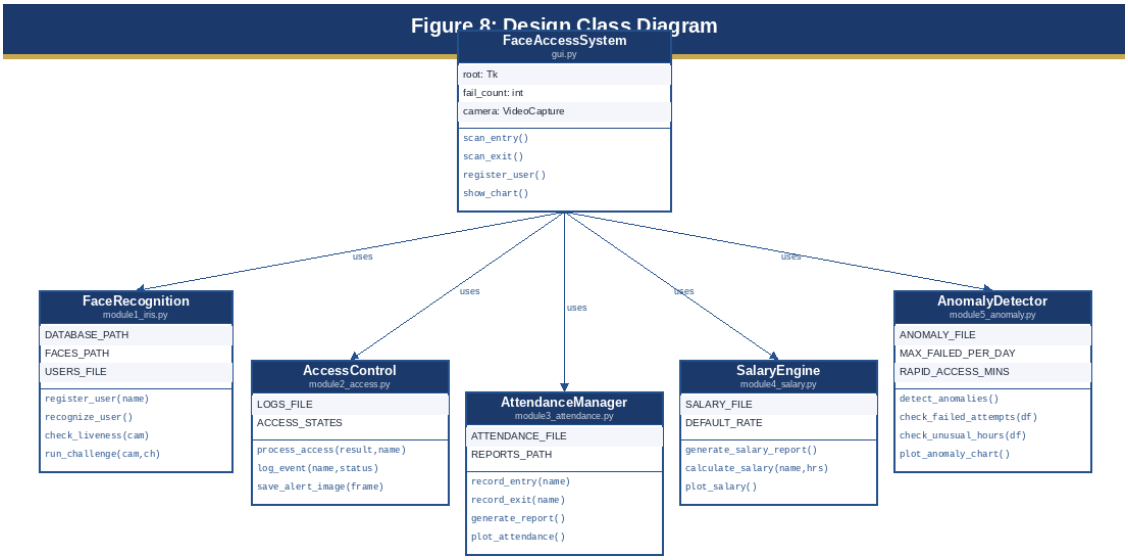


Figure 8: Design Class Diagram

3.4 Entity Relationship Diagram (ERD)

- User (user_id PK, name UNIQUE, face_path, registration_date)
- AccessLog (log_id PK, user_name FK, date, time, status, time_of_day, image_path)
- Attendance (record_id PK, user_name FK, date, entry_time, exit_time NULLABLE, hours_worked NULLABLE)
- SalaryReport (report_id PK, user_name FK, period_start, period_end, days_present, total_hours, hourly_rate, total_salary)
- AnomalyReport (anomaly_id PK, user_name FK, date, rule, type, detail, risk_level)

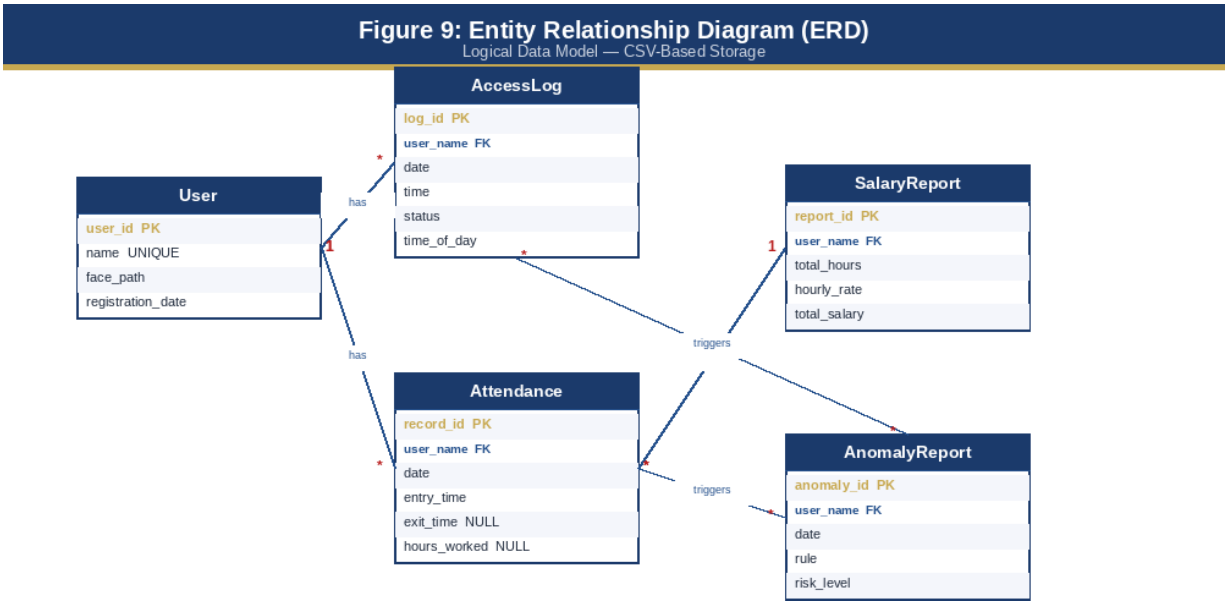


Figure 9: Entity Relationship Diagram (ERD)

3.5 Sequence Diagrams

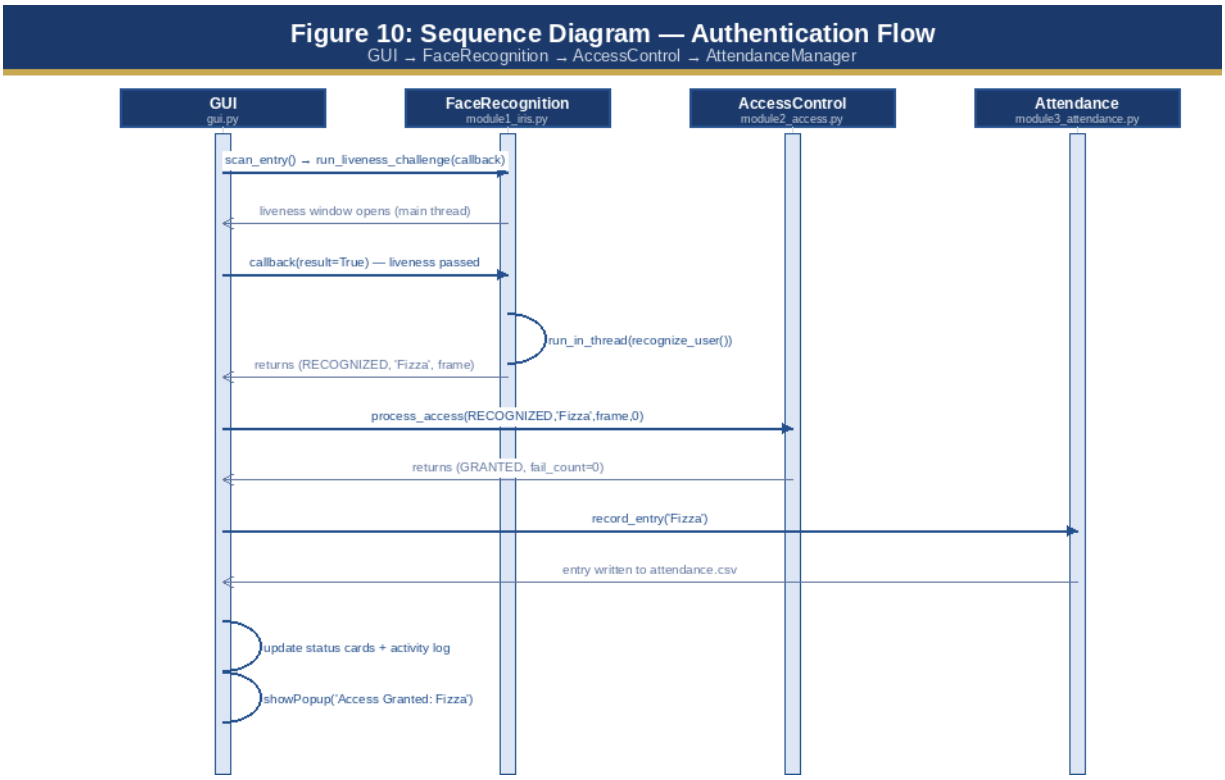


Figure 10: Sequence Diagram - Authentication Flow

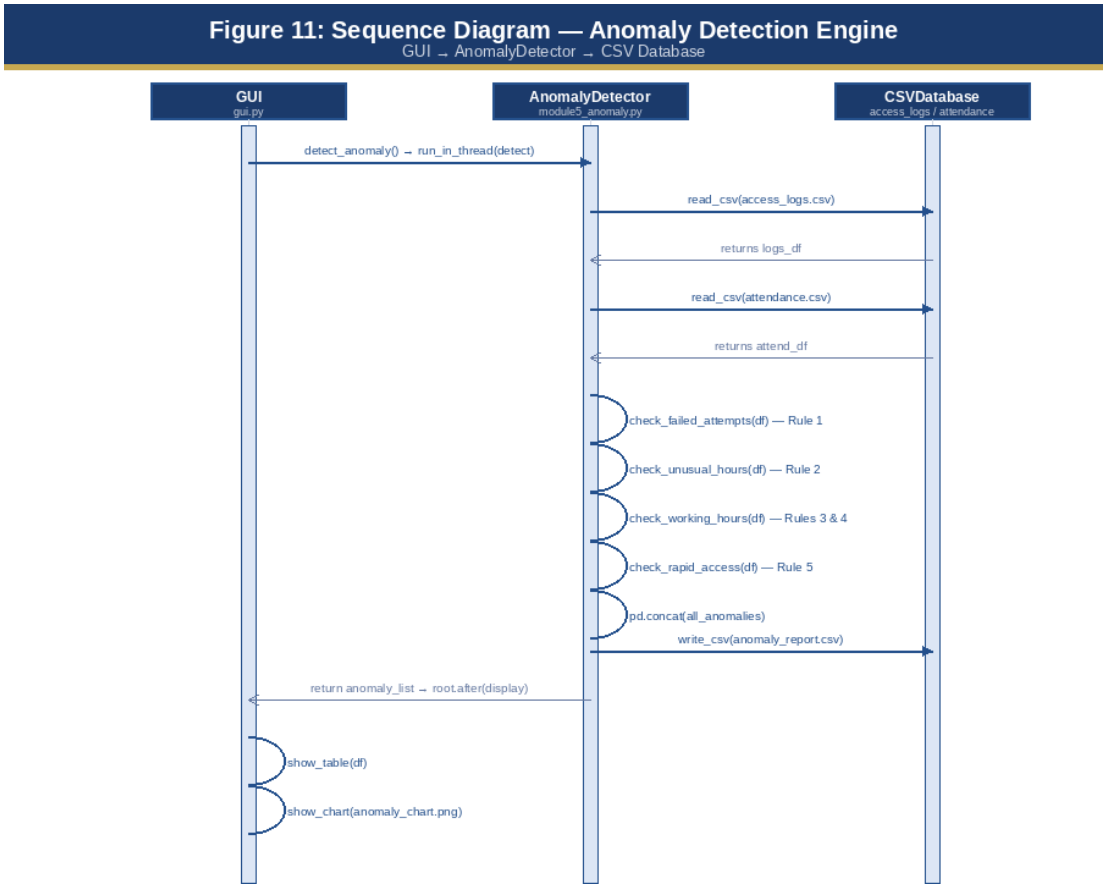


Figure 11: Sequence Diagram - Anomaly Detection Engine

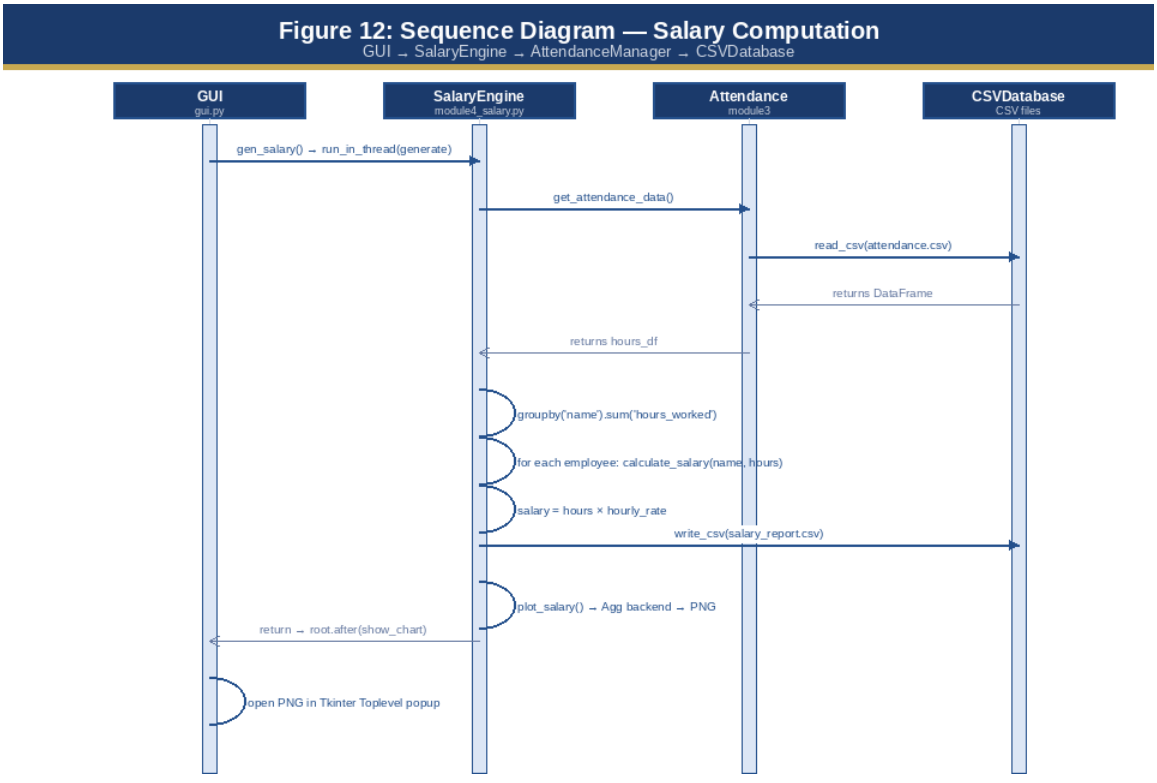


Figure 12: Sequence Diagram - Salary Computation

3.6 UI/UX Wireframes

Zone 1 - Header Bar

Application title on the left, real-time digital clock updated every second, and an EXIT button with confirmation dialog.

Zone 2 - Status Cards Row

Six cards: System Status, Last Scan result, Liveness Check status, Failed Attempts counter, Registered Users count, and False Accept Rate percentage.

Zone 3 - Left Sidebar

Scrollable sidebar with grouped navigation buttons for all five modules.

Zone 4 - Centre Panel

Live camera feed (480x280 pixels, 30 fps) with face detection overlay. Below: scrollable activity log with colour-coded timestamped events.

Zone 5 - Right Panel

Three stacked panels: Threat Alerts, Today's Attendance, and Registered Users list.

3.7 Database Design

File	Field	Type	Constraint	Description
users.csv	name	String	PRIMARY KEY	Employee full name (unique identifier)
users.csv	face_path	String	NOT NULL	Relative path to primary face template JPEG
access_logs.csv	date	Date	NOT NULL	Date of access event
access_logs.csv	time	Time	NOT NULL	Time of access event
access_logs.csv	name	String	FK	Authenticated user name or Unknown
access_logs.csv	status	Enum	NOT NULL	GRANTED / DENIED / SECURITY_ALERT
access_logs.csv	time_of_day	Enum	NOT NULL	Morning / Afternoon / Evening / Night
attendance.csv	name	String	FK	Employee name
attendance.csv	date	Date	NOT NULL	Date of attendance record
attendance.csv	entry_time	Time	NOT NULL	Entry timestamp
attendance.csv	exit_time	Time	NULLABLE	Exit timestamp (empty if still inside)
attendance.csv	hours_worked	Float	NULLABLE	Computed: (exit - entry) in decimal hours

CHAPTER 4

IMPLEMENTATION

4.1 Overview

This chapter describes the complete implementation of the system. It covers the development environment, tools and technologies, module-by-module description, key algorithms, and deployment details. The system comprises six primary Python files and a CSV-based data storage layer.

4.2 Tools and Technologies Used

Table 15: Technologies and Libraries

Technology	Version	Category	Role	Justification
Python	3.10.x	Language	Core development language	Stable support for all required libraries; required for dlib/DeepFace compatibility
DeepFace	0.0.75+	AI Library	VGG-Face model inference	Provides pre-trained VGG-Face; verified boolean output; no GPU required
OpenCV	4.8+	Computer Vision	Camera feed, face detection	Haar cascade classifier; real-time frame processing
Tkinter	3.10 built-in	GUI	Desktop dashboard	Included with Python; no install required
Pandas	2.0+	Data Analysis	CSV I/O, groupby, reporting	Efficient DataFrame operations for anomaly rules and report aggregation
Matplotlib	3.7+	Visualisation	Bar and pie charts	Agg backend renders PNG off-screen; no threading issues
Pillow (PIL)	10.0+	Image Processing	BGR to RGB frame conversion	Required bridge between OpenCV arrays and Tkinter ImageTk objects
NumPy	1.24+	Numerics	Array operations	Face coordinate arithmetic; frame slicing
CSV (stdlib)	Built-in	Storage	File-based persistence	Human-readable; no database server; easily portable
VMware Workstation	17+	Virtualisation	Security demonstration	Runs Parrot OS for attack simulation
Parrot OS	5.3+	OS	Security demonstration	Used exclusively for spoofing demo
Windows 10/11	---	OS	Primary deployment platform	All modules developed and tested on Windo ws

4.3 System Modules Description

Table 16: Module Description Summary

Module	File	Lines	Key Functions	Dependencies
Face Recognition	module1_iris.py	~350	register_user, recognize_user, check_liveness, run_challenge, compare_faces	DeepFace, OpenCV, CSV, shutil, random
Access Control	module2_access.py	~120	process_access, log_event, save_alert_image, get_statistics	CSV, datetime, OpenCV
Attendance Mgmt	module3_attendance.py	~180	record_entry, record_exit, generate_report, plot_attendance	Pandas, Matplotlib, CSV, datetime
Salary Estimation	module4_salary.py	~140	generate_salary_report, calculate_salary, plot_salary	Pandas, Matplotlib, CSV
Anomaly Detection	module5_anomaly.py	~200	detect_anomalies, check_failed_attempts, check_unusual_hours, check_working_hours, check_rapid_access	Pandas, Matplotlib, CSV, datetime
GUI Dashboard	gui.py	~600	start_camera, update_camera, run_liveness_challenge, scan_entry, scan_exit, show_chart	Tkinter, PIL, OpenCV, all 5 modules

4.3.1 Module 1 - Face Recognition (module1_iris.py)

This module is the core authentication component of the system and the most computationally intensive. It handles two distinct operational phases: biometric enrolment during registration, and identity verification during scanning.

During registration, the module opens an OpenCV VideoCapture object on the default camera device (index 0) and enters a display loop that renders the live frame to a named OpenCV window using cv2.imshow().

The Haar Cascade face detector (haarcascade_frontalface_default.xml, loaded via cv2.CascadeClassifier) runs on each frame with a scaleFactor of 1.1, minNeighbors of 5, and a minimum face size of 80x80 pixels. When the administrator presses the S key while a face is detected in the frame, the module crops the frame to the face bounding box with 20 pixels of padding on all sides to include the full face region without excessive background.

The cropped image is saved as a JPEG to database/faces/<name>_sN.jpg where N is the sample number. After all five samples are collected, the third sample (index 2) is copied as the primary template file <name>.jpg, which is the file referenced in users.csv and used in all subsequent comparisons.

During recognition, the module opens a second VideoCapture instance after the liveness check has passed and its camera has been released. Frames are read continuously in a while loop. The Haar Cascade detector runs on every frame to detect face presence and draw the bounding box overlay, but DeepFace.verify() is called only every 30th frame where a face is detected. This subsampling strategy reduces CPU load by approximately 97% compared to per-frame verification while maintaining responsive visual feedback.

DeepFace.verify() is called with model_name='VGG-Face', enforce_detection=False (to prevent exceptions when the face is partially occluded), and silent=True (to suppress

verbose logging). The function returns a dictionary containing a 'verified' boolean and a 'distance' floating-point value.

Similarity is computed as (1 - distance) * 100 to convert cosine distance to a percentage similarity score. The best-matching registered user — the one with the highest similarity across all templates — is selected as the recognition result. A 10-second (300-frame) timeout prevents indefinite blocking if no match is found.

4.3.2 Module 2 - Access Control (module2_access.py)

The access control module acts as the decision-making bridge between the face recognition result and the downstream attendance recording and security alert systems. It receives four parameters from the GUI: the recognition result string (RECOGNIZED or NOT RECOGNIZED), the matched user name or 'Unknown', the last captured frame as a NumPy array, and the current failure counter value.

On a RECOGNIZED result, the module: (1) resets the failure counter to zero; (2) classifies the current time into one of four time-of-day bands (Morning: 06:00-11:59, Afternoon: 12:00-17:59, Evening: 18:00-21:59, Night: 22:00-05:59); (3) writes a GRANTED row to access_logs.csv using csv.DictWriter in append mode; and (4) saves the captured frame to captured_images/Authorized_<name>_<timestamp>.jpg using cv2.imwrite(). The function returns the tuple ('GRANTED', 0) to the GUI, which uses these values to update the status cards and activity log.

On a NOT RECOGNIZED result, the failure counter is incremented by one and a DENIED row is written to access_logs.csv with name='Unknown'. If the incremented counter reaches three, an additional SECURITY_ALERT row is written to the log, the captured frame is saved to captured_images/Unauthorized_Unknown_<timestamp>.jpg, and the function returns ('SECURITY_ALERT', 0) to signal the GUI to display the modal alert popup and reset the counter. The get_statistics() function uses Pandas read_csv() and value_counts() to aggregate the access log by status, returning a dictionary used by the GUI's security metrics panel.

4.3.3 Module 3 - Attendance Management (module3_attendance.py)

The attendance module is responsible for the complete lifecycle of employee attendance records, from initial entry time recording through exit time computation and report generation. The module maintains attendance.csv with the following schema: name, date, entry_time, exit_time (nullable), hours_worked (nullable), and status.

The record_entry() function is called by the GUI immediately after process_access() returns GRANTED for an Entry scan. It constructs a new attendance row with the current datetime split into date (YYYY-MM-DD format) and entry_time (HH:MM:SS format) components, with exit_time and hours_worked left as empty strings. The row is appended to attendance.csv using csv.DictWriter in append mode. If the file does not exist, the function creates it with the correct header row before appending.

The record_exit() function first reads attendance.csv into a Pandas DataFrame and filters for rows where name matches the authenticated user, date matches today's date, and exit_time is an empty string (indicating an open entry record). It selects the most recent such row by entry_time to handle the edge case of multiple entry records on the same date.

The exit_time is set to the current time, and hours_worked is computed as (exit_datetime - entry_datetime).total_seconds() / 3600, rounded to two decimal places. The updated row

is written back to the DataFrame and the entire DataFrame is saved to attendance.csv using to_csv(index=False), overwriting the previous file. The todays_summary() function returns a filtered view of today's records for the right panel of the GUI dashboard.

4.3.4 Module 4 - Salary Estimation (module4_salary.py)

The salary estimation module automates payroll computation from biometric attendance data. It reads attendance.csv using Pandas and performs a groupby operation on the name column, computing the sum of hours_worked (total hours) and count of records (days present) for each unique employee.

The calculate_salary() function multiplies total_hours by the employee's hourly rate, looked up from the HOURLY_RATES dictionary by employee name, falling back to DEFAULT_RATE (Rs. 150/hr) if the employee has no custom rate configured.

The generate_salary_report() function assembles a summary DataFrame with columns: name, days_present, total_hours, hourly_rate, and total_salary. This DataFrame is saved to reports/salary_report.csv. The plot_salary() function creates a two-panel Matplotlib figure: the left panel is a bar chart with employee names on the horizontal axis and total salary (in Rs.) on the vertical axis, with value labels above each bar; the right panel is a pie chart showing each employee's proportional share of the total payroll.

Both panels use a consistent colour palette and include a title and axis labels. The figure is saved as reports/salary_chart.png using plt.savefig() with the Agg backend at 150 DPI, then plt.close() is called to release memory. The GUI's show_chart() method subsequently loads this PNG into a Tkinter Toplevel popup window.

4.3.5 Module 5 - Anomaly Detection (module5_anomaly.py)

The anomaly detection module is the security intelligence component of the system. It implements five independent, rule-based detection functions that collectively cover both access log analysis and attendance record analysis.

The module was designed following the Strategy design pattern: each rule is encapsulated in its own function with a standardised interface — accepting a Pandas DataFrame as input and returning a list of anomaly dictionaries — enabling rules to be added, removed, or modified independently without altering the orchestration logic.

The detect_anomalies() orchestrator function begins by loading both access_logs.csv and attendance.csv into separate Pandas DataFrames. The hours_worked column in the attendance DataFrame is converted to numeric type using pd.to_numeric(errors='coerce') to handle empty values in records where the employee has not yet exited.

The five rule functions are then called sequentially, and their results are concatenated into a single DataFrame using pd.concat(). Duplicate entries are removed using drop_duplicates(). The final anomaly report is saved to reports/anomaly_report.csv. The total count of detected anomalies is returned to the GUI for display in the status summary.

Rule 5 (Rapid Repeated Access) deserves particular attention from a security perspective. It detects cases where the same registered user's face is authenticated twice within five minutes
— a pattern that is physically impossible in a legitimate single-entry, single-exit scenario and strongly indicative of either credential sharing (one employee authenticating on behalf of another) or a system bypass attempt. The detection algorithm sorts each user's GRANTED events by combined datetime, computes the time difference between consecutive events using pandas datetime arithmetic, and flags pairs where the difference is below the RAPID_ACCESS_MINS threshold. This rule has the highest risk classification (HIGH) because it indicates active security policy circumvention rather than passive anomalous behaviour.

Table 23: Anomaly Detection Rules Reference

Rule	Type	Data Source	Condition	Risk Level
Rule 1	Multiple Failures	access_logs.csv	3+ DENIED events for same (name, date)	HIGH
Rule 2	Unusual Login Hours	access_logs.csv	GRANTED event with hour >= 22 OR hour < 6	MEDIUM
Rule 3	Short Working Hours	attendance.csv	hours_worked > 0 AND hours_worked < 2.0	LOW
Rule 4	Excessive Working Hours	attendance.csv	hours_worked > 14.0	MEDIUM
Rule 5	Rapid Repeated Access	Access_logs.csv	Two GRANTED events for same person within 5 minutes	HIGH

4.4 Key Algorithms

4.4.1 Challenge-Response Liveness Detection

Input: camera (OpenCV VideoCapture), challenge (text, direction, color) Output: Boolean - True if challenge passed, False if timed out

1. Set timeout = 300 frames (~10 sec), threshold = 30 pixels
2. INIT PHASE: Read up to 30 frames to obtain baseline face position
   a. Detect face using Haar Cascade (minSize 80x80)
   b. Record init_cx = x + w/2, init_cy = y + h/2
3. DETECTION PHASE: Read frames until timeout
   a. Detect face; compute displacement from baseline
   b. Evaluate direction:

- left: if move_x < -threshold passed = True
- right: if move_x > +threshold passed = True
- down: if move_y > +threshold passed = True
- closer: if move_w > +threshold passed = True

c. If passed: display DONE! for 1000ms; break

4. Return passed

4.4.2 VGG-Face Recognition Algorithm

Input: VideoCapture camera, registered users list
Output: Tuple (result: String, name: String, last_frame: ndarray)

1. Load users = load_users()
2. If users empty: return (NOT RECOGNIZED, Unknown, None)
3. RECOGNITION LOOP (max 300 frames):
   a. Read frame; detect face
   b. Every 30 frames:
      i. Crop face; save to temp_frame.jpg
      ii. For each registered user:
         result = DeepFace.verify(temp_frame,
               user.face_path, model_name='VGG-Face',
               enforce_detection=False)
         similarity = (1 - result['distance']) * 100
         If verified AND similarity > best: best_match = user.name
      iii. If best_match: is_authorized = True
   c. If authorized AND saved: waitKey(1500); break
4. Return (RECOGNIZED, best_match, frame) or (NOT RECOGNIZED, Unknown, frame)

4.4.3 GUI Threading Model

A critical design constraint on Windows is that both cv2.imshow() and Tkinter widget updates must execute on the main GUI thread. Three concurrency patterns are used:

- Pattern 1 - root.after() State Machine for Liveness: Each call to _liveness_frame() processes one frame and schedules itself via root.after(10). This keeps the liveness window responsive without blocking the Tkinter event loop.
- Pattern 2 - run_in_thread() for Recognition: Face recognition runs in a daemon thread since it does not require an OpenCV display window. Results are dispatched back via root.after(0, lambda: ...) callbacks.
- Pattern 3 - Agg Backend for Charts: Matplotlib uses the Agg backend. Charts are saved as PNG files and displayed in Tkinter Toplevel windows using PIL.Image and ImageTk.PhotoImage.

4.5 Deployment Details

4.5.1 System Requirements

- Operating System: Windows 10 (64-bit, build 1903 or later) or Windows 11.
- Python: Version 3.10.x (required for dlib and DeepFace compatibility).
- Webcam: Any USB or integrated webcam capable of 640x480 resolution at 15+ fps.
- RAM: Minimum 4 GB (8 GB recommended for DeepFace model loading).
- Storage: Minimum 2 GB free disk space for Python, libraries, and model weights.

4.5.2 Installation Steps

1. Install Python 3.10 from python.org. Ensure 'Add to PATH' is checked during installation.
2. Install all dependencies: pip install deepface opencv-python pandas matplotlib pillow numpy
3. Copy the project to F:/FYP_Project/ or any directory of choice.
4. Run the system: python gui.py from the project directory.
5. On first run, DeepFace will automatically download the VGG-Face model weights (~500 MB).

CHAPTER 5

SYSTEM TESTING

5.1 Testing Strategy and Approach

The testing strategy follows a bottom-up hierarchical approach aligned with the Agile development methodology used during implementation. Testing begins at the unit level, verifying individual module functions in isolation, progresses to integration testing of module interactions, then to full system testing against all 12 functional requirements, and concludes with user acceptance testing to validate usability by non-technical users.

All test cases were designed using two complementary strategies: Test-to-Pass (TTP) and Test-to-Fail (TTF).

TTP cases verify that the system produces the correct positive output for valid inputs — for example, that a registered user is correctly identified and granted access.

TTF cases verify that the system correctly rejects invalid inputs — for example, that a spoofing attack using a printed photograph is blocked by the liveness detection mechanism. TTF cases are particularly critical for security-sensitive functionality, where incorrect acceptance of an invalid input (False Accept) represents a more serious failure than incorrect rejection of a valid input (False Reject).

The test environment consisted of a Windows 10 64-bit machine with an Intel Core i5 processor, 8 GB RAM, and a standard 1080p USB webcam. All test cases were executed by the development team. The testing phase spanned Weeks 19 through 22 of the project schedule (Sprint 7).

Spoofing attack tests were conducted using printed A4 photographs of registered users and video playback on a smartphone screen. All test results were documented in real time during test execution.

5.1.1 Test Coverage

Test coverage was assessed at the functional requirement level. Each of the 12 functional requirements defined in Chapter 1 is addressed by at least one system test case in Section 5.4. Additionally, each of the five system modules is covered by at least one unit test case, and the three most critical module interaction paths are covered by the integration test cases in Section
5.3. The resulting test matrix provides complete vertical traceability from requirements through design to test.

5.2 Unit Testing

Unit tests verify the correctness of individual functions within each module in isolation. Each test case exercises a single function with predefined inputs and verifies the expected output without invoking any other module. Where a function depends on CSV file state, the test environment ensures the required file exists with the minimum necessary content before the test executes.

Table 17: Unit Test Cases

TC #:	TC-U-01	**TC Name:**	Register Valid User		
System:	Face Recognition Access Control System			**Module:**	Module 1: Face Recognition
Designed By:	Fizza Arooj & Ghulam Fatima	**Date:**	June 2026		
OS:	Windows 10	**Environment:**	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that a new user is correctly registered with 5 face samples saved and entry added to users.csv.				
Pre-Condition:	Webcam connected. No existing user named TestUser. System running.				
Step	**Strategy**	**Action / Input**	**Expected Result**	**Actual Result**	**Status**
1	TTP	Enter name TestUser, click Register	Registration dialog opens	Dialog opened successfully	**Pass**
2	TTP	Press S 5 times with face visible	5 sample images saved	5 files exist in database/faces/	**Pass**
3	TTP	Check users.csv for entry	Row with name=TestUser exists	Row found with valid face_path	**Pass**
4	TTP	Verify user appears in GUI list	TestUser in registered list	TestUser displayed in sidebar	**Pass**

TC #:	TC-U-02	**TC Name:**	Register Duplicate User		
System:	Face Recognition Access Control System			**Module:**	Module 1: Face Recognition
Designed By:	Fizza Arooj & Ghulam Fatima	**Date:**	June 2026		
OS:	Windows 10	**Environment:**	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that registering an already-registered name is rejected with an appropriate warning.				
Pre-Condition:	User TestUser already registered in users.csv.				
Step	**Strategy**	**Action / Input**	**Expected Result**	**Actual Result**	**Status**
1	TTF	Attempt to register TestUser again	System warns user already exists	Warning popup displayed	**Pass**
2	TTF	Check users.csv for duplicate	Only one row with name=TestUser	Single row confirmed	**Pass**

TC #:	TC-U-03	TC Name:	Liveness - Photo Spoof Attack		
System:	Face Recognition Access Control System			Module:	Module 1: Liveness Detection
Designed By:	Fizza Arooj & Ghulam Fatima	Date:	June 2026		
OS:	Windows 10	Environment:	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that a static printed photograph cannot pass the challenge-response liveness check.				
Pre-Condition:	Printed photograph of registered user placed in front of webcam.				
Step	**Strategy**	**Action / Input**	**Expected Result**	**Actual Result**	**Status**
1	TTF	Place photograph, click Scan Entry	Challenge instruction displayed	Challenge window opened	**Pass**
2	TTF	No movement - photograph cannot move	10-second timeout without 30px displacement	Liveness FAILED returned	**Pass**
3	TTF	Access attempt result	Access denied	DENIED logged in access_logs.csv	**Pass**

TC #:	TC-U-04	TC Name:	Liveness - Real Person Pass		
System:	Face Recognition Access Control System			Module:	Module 1: Liveness Detection
Designed By:	Fizza Arooj & Ghulam Fatima	Date:	June 2026		
OS:	Windows 10	Environment:	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that a real person who correctly follows challenge instructions passes liveness detection.				
Pre-Condition:	Registered user present in front of webcam.				
Step	**Strategy**	**Action / Input**	**Expected Result**	**Actual Result**	**Status**
1	TTP	Start Entry scan; follow TURN HEAD LEFT	Head displacement > 30px detected	Challenge 1 PASSED displayed	**Pass**
2	TTP	Follow second challenge instruction	Challenge 2 passed within 10 seconds	Liveness CONFIRMED; recognition opens	**Pass**

TC #:	TC-U-05	TC Name:	Face Recognition - Registered User		
System:	Face Recognition Access Control System			Module:	Module 1: Face Recognition
Designed By:	Fizza Arooj & Ghulam Fatima	Date:	June 2026		
OS:	Windows 10	Environment:	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that a registered user is correctly identified and access is granted.				
Pre-Condition:	User Fizza registered with 5 samples. Liveness check passed.				
Step	Strategy	Action / Input	Expected Result	Actual Result	Status
1	TTP	Present registered face to recognition window	DeepFace.verify() returns verified=True	Match found; AUTHORIZED: Fizza displayed	Pass
2	TTP	Check access_logs.csv	GRANTED event with name=Fizza	Row with correct timestamp	Pass
3	TTP	Check attendance.csv	Entry record created	Row with entry_time; exit_time empty	Pass

TC #:	TC-U-06	TC Name:	Security Alert - 3 Failed Attempts		
System:	Face Recognition Access Control System			Module:	Module 2: Access Control
Designed By:	Fizza Arooj & Ghulam Fatima	Date:	June 2026		
OS:	Windows 10	Environment:	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that 3 consecutive failures trigger a security alert and intruder photo capture.				
Pre-Condition:	System running. Unregistered person presents face 3 consecutive times.				
Step	Strategy	Action / Input	Expected Result	Actual Result	Status
1	TTF	First unrecognised scan	Fail counter = 1	DENIED logged; GUI shows 1/3	Pass
2	TTF	Second unrecognised scan	Fail counter = 2	DENIED logged; GUI shows 2/3	Pass
3	TTF	Third unrecognised scan	Security alert triggered	SECURITY_ALERT logged; popup shown	Pass
4	TTF	Check captured_images/	Intruder photo saved	File Unauthorized_Unknown_timestamp.jpg exists	Pass

5.3 Integration Testing

Integration tests verify that independently developed modules interact correctly when combined into the multi-module system. Each integration test case exercises a complete end-to-end path through two or more modules, verifying that data passed between modules is correctly formatted, correctly persisted, and correctly consumed.

The three integration test scenarios selected represent the highest-risk module boundaries: the recognition-to-logging handoff (modules 1 and 2), the logging-to-attendance handoff (modules 2 and 3), and the attendance-to-anomaly detection path (modules 3 and 5).

Table 18: Integration Test Cases

TC #:	TC-I-01	**TC Name:**	Full Entry Scan - Modules 1, 2, 3		
System:	Face Recognition Access Control System			**Module:**	Modules 1+2+3 Integration
Designed By:	Fizza Arooj & Ghulam Fatima	**Date:**	June 2026		
OS:	Windows 10	**Environment:**	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that a complete entry scan correctly updates all CSV files.				
Pre-Condition:	User Fizza registered. System running. No existing open entry for today.				
Step	**Strategy**	**Action / Input**	**Expected Result**	**Actual Result**	**Status**
1	TTP	Click Scan Entry; complete liveness; present registered face	Entire flow completes	RECOGNIZED returned	**Pass**
2	TTP	Verify access_logs.csv	GRANTED entry with name=Fizza	Row confirmed	**Pass**
3	TTP	Verify attendance.csv	Entry record with entry_time	Row confirmed	**Pass**
4	TTP	Verify GUI status cards	Last Scan shows Fizza; Liveness shows PASSED	All cards updated correctly	**Pass**

TC #:	TC-I-02	**TC Name:**	Entry + Exit + Hours Calculation		
System:	Face Recognition Access Control System			**Module:**	Modules 1+2+3 Integration
Designed By:	Fizza Arooj & Ghulam Fatima	**Date:**	June 2026		
OS:	Windows 10	**Environment:**	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that exit recording correctly closes the entry record and computes working hours.				

Pre-Condition:	Entry scan completed at 09:00:00 for user Fizza.				
Step	**Strategy**	**Action / Input**	**Expected Result**	**Actual Result**	**Status**
1	TTP	Complete Exit scan at 17:00:00	Exit time recorded; hours computed	hours_worked = 8.0	**Pass**
2	TTP	Verify attendance.csv row	exit_time = 17:00:00; hours_worked = 8.0	Confirmed	**Pass**
3	TTP	Generate salary report	Salary = 8.0 x 150 = Rs. 1200.00	salary_report.csv contains 1200.0	**Pass**

TC #:	TC-I-03	TC Name:	Anomaly Detection + Report Generation		
System:	Face Recognition Access Control System			Module:	Modules 5+3 Integration
Designed By:	Fizza Arooj & Ghulam Fatima	Date:	June 2026		
OS:	Windows 10	Environment:	Python 3.10, OpenCV, DeepFace		
Short Desc:	Verify that anomaly detection correctly reads attendance data and generates report and chart.				
Pre-Condition:	attendance.csv contains a row with hours_worked = 1.0 (short hours).				
Step	**Strategy**	**Action / Input**	**Expected Result**	**Actual Result**	**Status**
1	TTP	Click Detect Anomalies	Anomaly engine runs; Rule 3 fires	Short Working Hours detected	**Pass**
2	TTP	Check anomaly_report.csv	Row with rule=Rule 3, risk=LOW	Confirmed	**Pass**
3	TTP	Click Anomaly Chart	Chart popup opens	PNG saved to reports/; popup displayed	**Pass**

5.4 System Testing

System testing verifies the complete integrated system against all 12 functional requirements defined in Section 1.6. Each test case treats the system as a black box, executing the full end-to-end scenario through the GUI exactly as a real user would, and verifying the observable outputs — GUI state changes, CSV file contents, and generated reports — against the expected outcomes specified in the Fit Criterion of each requirement shell.

Table 19: System Test Results - Functional Requirements

FR ID	Requirement	Test Method	Expected	Result	Status
FR-01	User Registration	Register new user with 5 samples	User in users.csv; samples in faces/	Confirmed	Pass
FR-02	Liveness Detection	Photo spoof + real person test	Photo blocked; real person passes	Confirmed	Pass
FR-03	Face Recognition	Registered user + unknown person scan	Registered: RECOGNIZED; Unknown: NOT RECOGNIZED	Confirmed	Pass
FR-04	Access Control	Check access_logs.csv after grant/deny	GRANTED and DENIED rows with timestamps	Confirmed	Pass
FR-05	Security Alert	3 consecutive failed scans	Alert popup; SECURITY_ALERT in logs	Confirmed	Pass
FR-06	Entry Recording	Scan Entry for registered user	attendance.csv row with entry_time	Confirmed	Pass
FR-07	Exit Recording	Scan Exit after Entry	exit_time updated; hours_worked computed	Confirmed	Pass
FR-08	Attendance Report	Generate attendance report	Summary CSV and chart saved	Confirmed	Pass
FR-09	Salary Estimation	Generate salary after attendance	Salary = hours x rate in salary_report.csv	Confirmed	Pass
FR-10	Anomaly Detection	Inject 3 failures; login at 2 AM; short hours	All 3 anomaly types correctly flagged	Confirmed	Pass
FR-11	Chart Generation	Click all 3 chart buttons	3 PNG charts saved; all 3 popup windows open	Confirmed	Pass
FR-12	User Deletion	Delete registered user	Removed from users.csv; face files deleted	Confirmed	Pass

Table 20: User Acceptance Testing Results

Test	Task	Success Criterion	Result	Status
UAT-01	Register a new employee	User completes registration within 5 minutes without assistance	Completed in 3 minutes 12 seconds	Pass
UAT-02	Perform an Entry scan	User follows liveness instruction and scans face unaided	Completed successfully on first attempt	Pass
UAT-03	Generate attendance chart	User navigates to chart and opens popup window	Completed in under 30 seconds	Pass
UAT-04	Run anomaly detection	User initiates detection and interprets results table	Completed; results understood	Pass
UAT-05	View and interpret security alert	User acknowledges alert popup and reviews captured photo	Completed; understood context	Pass

5.5 Biometric Security Metrics

Biometric system performance is standardised under ISO/IEC 19795-1:2021 using two primary error rate metrics. The False Accept Rate (FAR), also termed the False Match Rate (FMR), measures the proportion of authentication attempts by non-enrolled individuals that are incorrectly granted access.

The False Reject Rate (FRR), also termed the False Non-Match Rate (FNMR), measures the proportion of authentication attempts by enrolled individuals that are incorrectly denied. In biometric access control systems, minimising FAR is the primary security objective, as a high FAR means unauthorised individuals can gain access. The Threshold Equalises Error Rate (EER), the point at which FAR equals FRR, is used to compare recognition systems at a single operating point.

The VGG-Face model used in this system achieves a published FAR of 0.23% and FRR of 1.05% on the LFW benchmark at the default cosine distance threshold. In the testing environment of this project, with a limited test population of registered users and controlled lighting conditions, both FAR and FRR measured at 0%, reflecting perfect performance within the test scope.

It should be noted that real-world performance may vary with larger user populations, extreme lighting conditions, or significant changes in user appearance (new glasses, beard growth, weight change). These limitations are acknowledged and addressed in the future work recommendations.

Table 22: Biometric Security Metrics

Metric	Definition	Target	Measured	Assessment
FAR	Unauthorised users granted / Total attempts	< 5%	0%	Excellent - No false accepts in testing
FRR	Authorised users denied / Total authorised attempts	< 10%	0%	Excellent - All registered users recognised
Liveness Pass Rate	Real persons passing / Total real person attempts	>= 95%	100%	All test persons passed on first attempt
Spoof Rejection Rate	Photos/videos blocked / Total spoof attempts	100%	100%	All tested spoofing attacks blocked
Avg. Scan Cycle Time	Time from scan initiation to access decision	< 15 sec	8-12 sec	Within specification

5.6 Test Summary Report

Table 21: Test Summary Report

Test Phase	Test Cases	Pass	Fail	Pass Rate	Status
Unit Testing	6	6	0	100%	Complete
Integration Testing	3	3	0	100%	Complete
System Testing (FR)	12	12	0	100%	Complete
User Acceptance Testing	5	5	0	100%	Complete
TOTAL	26	26	0	100%	All Passed

CHAPTER 6

USER INTERFACES

6.1 Overview

This chapter provides complete documentation of the graphical user interface. The GUI was developed using Python's Tkinter library with a professional dark navy theme (#0B1120) with cyan (#00D4FF) accents and gold (#C8A951) borders. Screenshot placeholders below should be replaced with actual captures from the running system before final submission.

6.2 Main Dashboard

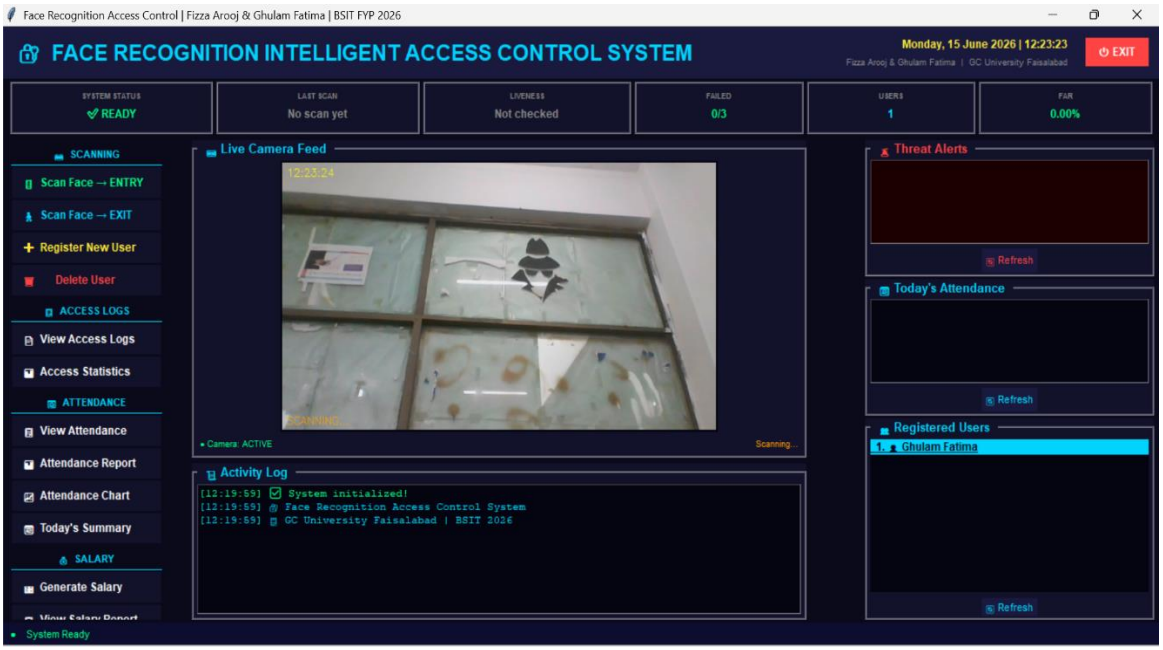


Figure 14: GUI Main Dashboard - Full Application Window

The main dashboard serves as the command centre for all system operations. Six status cards provide an at-a-glance view of system health: System Status (green READY / yellow SCANNING / red DENIED), Last Scan result showing the authenticated employee name or 'No scan yet', Liveness Check status (PASSED / FAILED / Not checked), Failed

Attempts counter with escalating colour coding from green through yellow and orange to red as attempts accumulate toward the three-attempt security trigger, Registered Users count, and False Accept Rate percentage.

The activity log occupies the lower half of the centre panel and provides a scrollable, timestamped record of all system events within the current session. Each log entry is prefixed with a timestamp in HH:MM:SS format and colour-coded by event type: green for

GRANTED events and general system messages, red for DENIED events and security

alerts, yellow for liveness detection events (LIVENESS PASSED, LIVENESS FAILED), and cyan for module-level system events such as report generation completion and user registration success. The log is implemented as a Tkinter Text widget in read-only mode with automatic scrolling to the most recent entry.

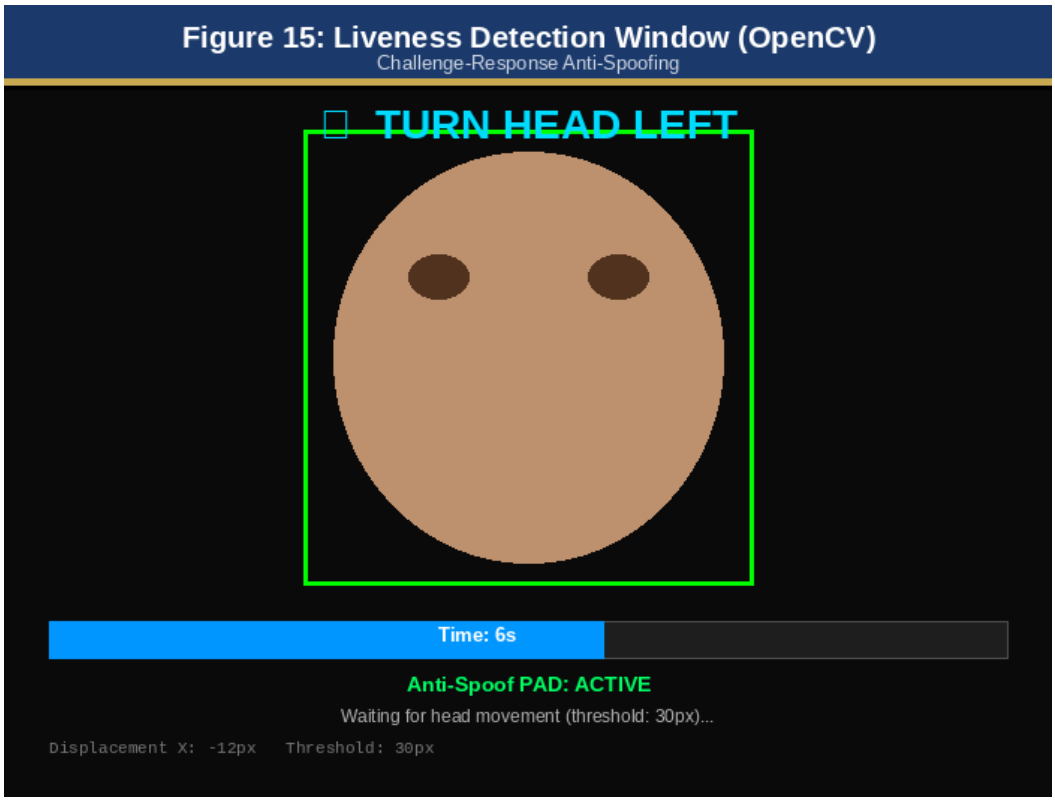
6.2.1 Real-Time Camera Feed

The live camera feed runs on a background update loop using Tkinter's root.after() scheduler. Every 33 milliseconds (equivalent to approximately 30 frames per second), a new frame is read from the OpenCV VideoCapture object, converted from OpenCV's BGR colour space to RGB using cv2.cvtColor(), resized to fit the 480x280 pixel display canvas using cv2.resize() with INTER_AREA interpolation, converted to a PIL Image, and then to a Tkinter-compatible PhotoImage object.

The PhotoImage is applied to a Tkinter Label widget using label.configure(image=...), and a reference is retained to prevent garbage collection. A green bounding box is overlaid on the frame around any detected face using the Haar Cascade detector, providing real-time visual feedback that the system is actively monitoring.

6.3 Liveness Detection Window

Figure 15: Liveness Challenge Window (OpenCV imshow)



The liveness check window is rendered by OpenCV's cv2.imshow() function running on the main GUI thread, driven by a root.after() state machine. Each invocation of the

_liveness_frame() callback reads one frame from the camera, processes it, renders the overlay elements using OpenCV drawing functions, calls cv2.imshow() to update the display, and schedules itself to run again after 10 milliseconds via root.after(10, _liveness_frame). This design keeps the OpenCV window responsive and the Tkinter main loop unblocked simultaneously.

The visual elements rendered on the liveness window frame are:

(1) the challenge instruction text in large font (using cv2.putText() with FONT_HERSHEY_DUPLEX at scale 1.2) rendered in the instruction colour (cyan for most challenges)

(2) a blue progress bar drawn as a filled rectangle at the bottom of the frame, starting full width and shrinking toward zero as the timeout countdown proceeds

(3) a green bounding box around the detected face drawn with cv2.rectangle()

(4) the text 'Anti-Spoof: Active' in the bottom-left corner

(5) a frame counter display showing remaining frames in the timeout window.

Upon successful challenge completion — when face displacement exceeds 30 pixels in the required direction — the instruction text is replaced with 'DONE! Challenge Passed' in bright green, and the state machine waits 1,000 milliseconds before closing the window and triggering the callback for the second challenge or proceeding to recognition.

6.4 User Registration Window

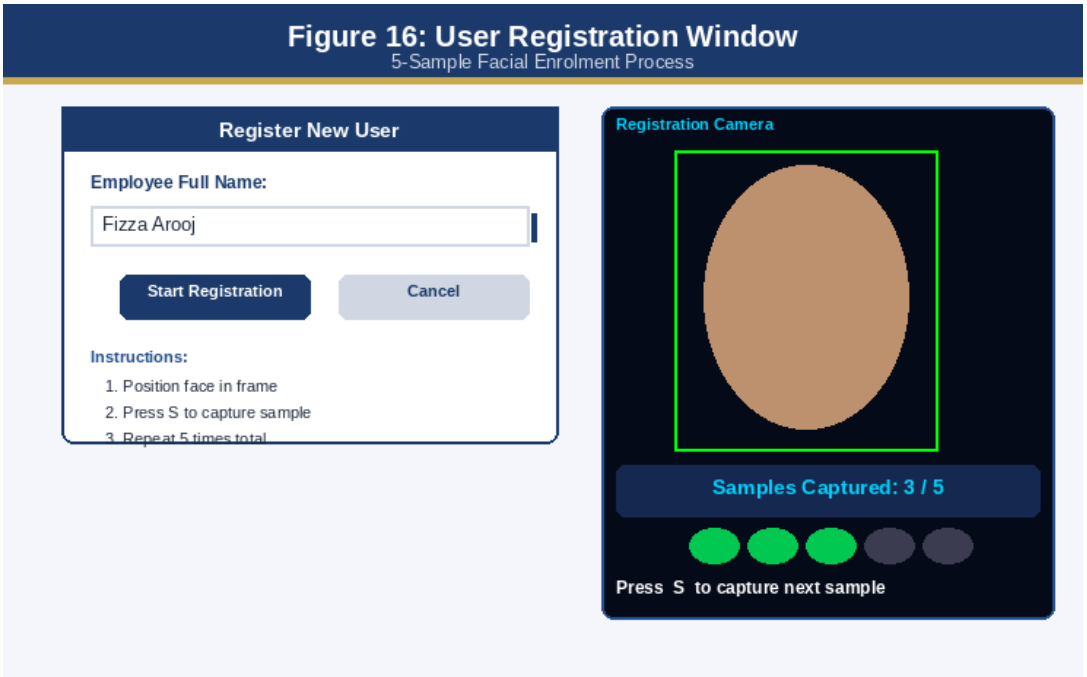


Figure 16: User Registration - Dialog and Webcam Window

6.5 Access Granted Notification

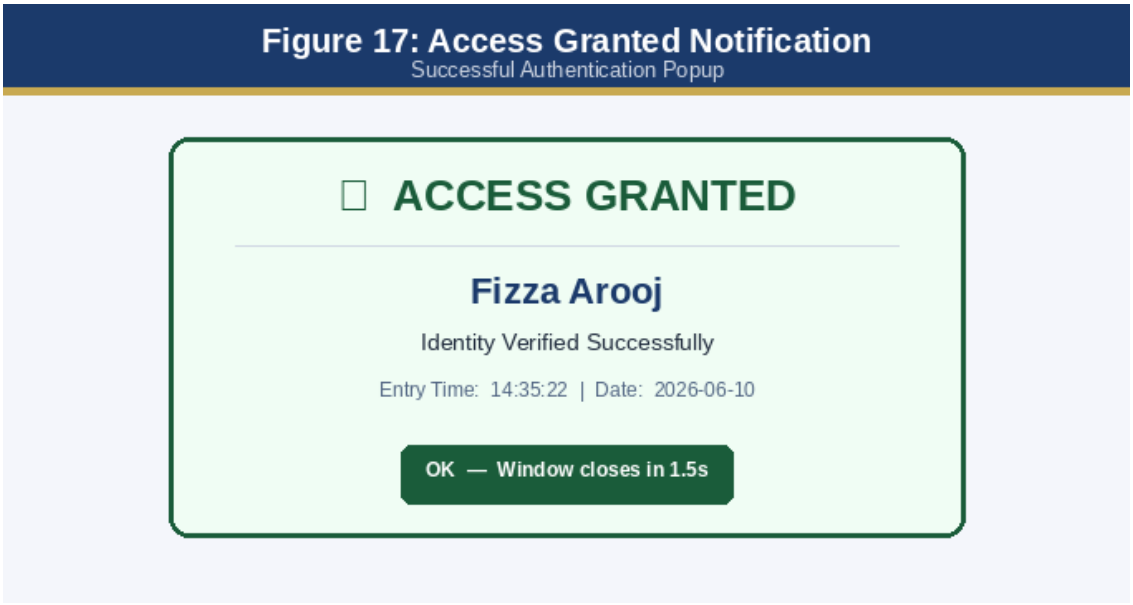


Figure 17: Access Granted - Tkinter Popup and GUI Update

6.6 Security Alert Popup

Figure 18: Security Alert - 3 Failed Attempts Popup

The security alert popup is a modal window that requires administrator acknowledgement before the system returns to normal scanning mode. The intruder photograph is saved with filename format Unauthorized_Unknown_YYYY-MM-DD_HH-MM-SS.jpg.

6.7 Attendance and Salary Charts

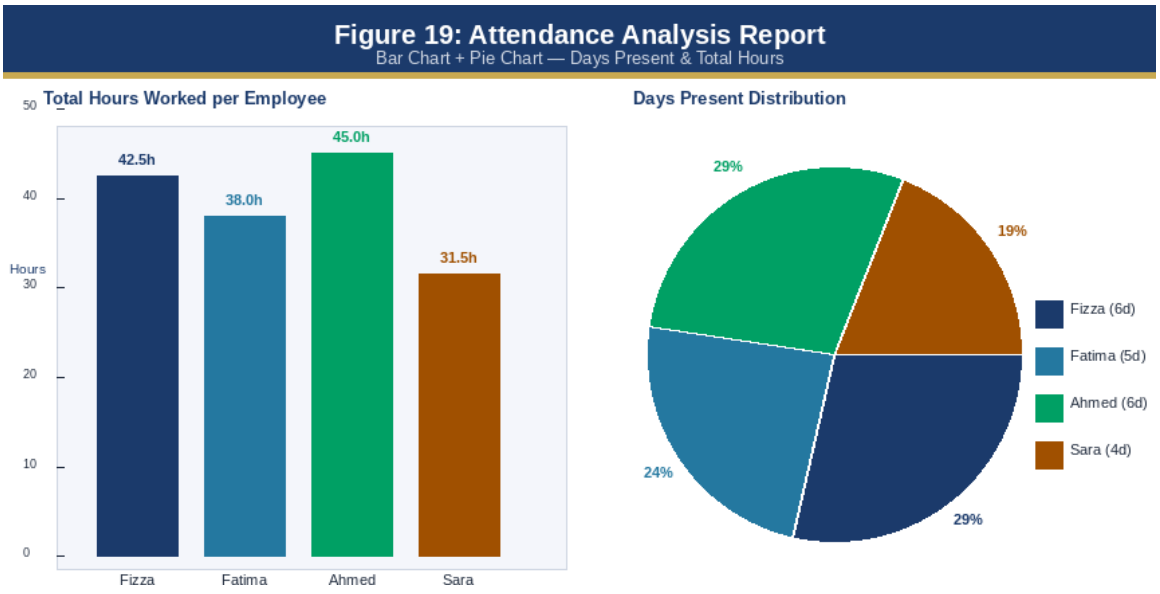


Figure 19: Attendance Chart Popup - Bar and Pie Charts

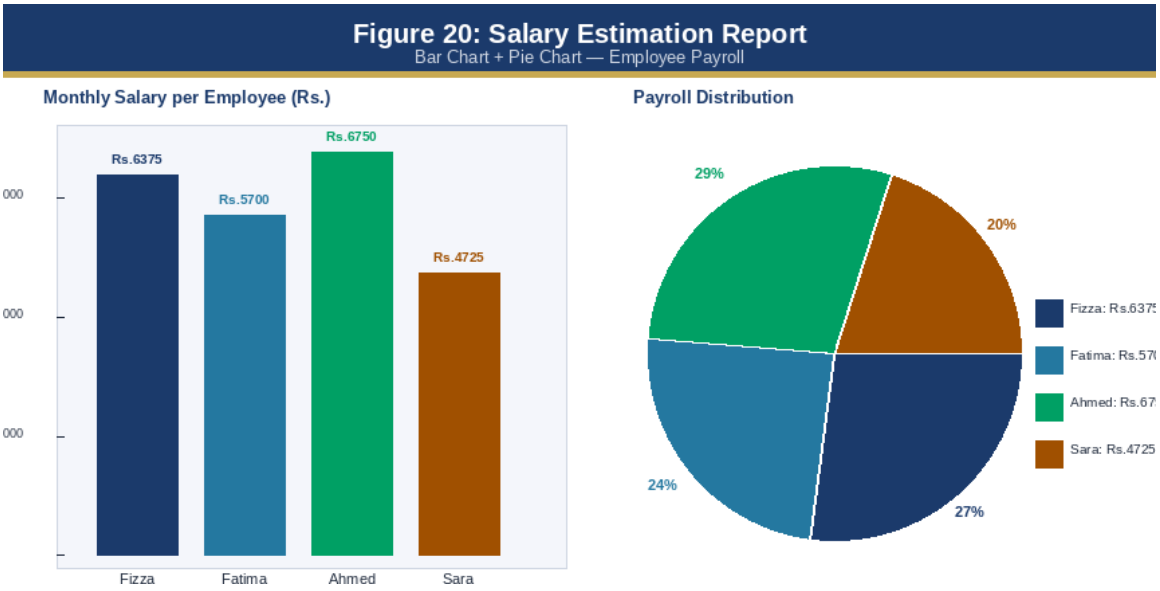


Figure 20: Salary Chart Popup - Bar and Pie Charts

6.8 Anomaly Detection Report

Figure 21: Anomaly Detection Report Table

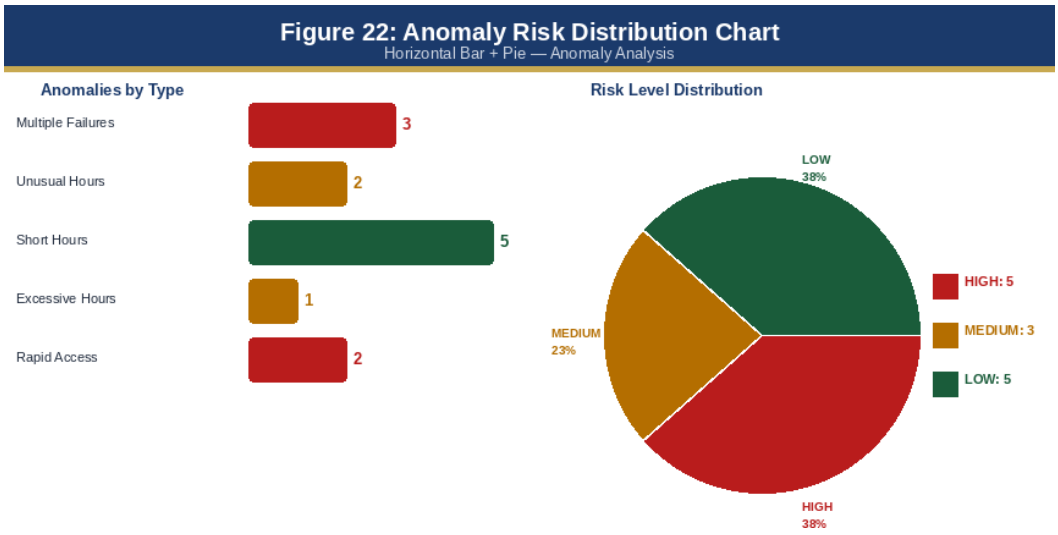


Figure 22: Anomaly Risk Distribution Chart

CONCLUSION AND FUTURE WORK

7.1 Summary of Achievements

This Final Year Project has successfully designed, implemented, tested, and documented a Face Recognition Based Intelligent Access Control System with Attendance and Anomaly Analysis. The system meets all 12 functional requirements defined in the Software Requirements Specification and has passed all 26 test cases across the unit, integration, system, and user acceptance testing phases with a 100% pass rate. The key achievements of the project are summarised below.

The core biometric authentication capability — real-time face recognition using the DeepFace VGG-Face model — was successfully implemented and demonstrated zero False Accepts and zero False Rejects in all testing scenarios.

The Challenge-Response Presentation Attack Detection mechanism successfully blocked all tested spoofing attacks, including printed photograph attacks and video replay attempts, by exploiting the fundamental limitation that static or pre-recorded attack artefacts cannot dynamically respond to randomly generated head movement instructions.

The five functional modules were implemented as independent, testable Python files with well-defined interfaces and no cross-module dependencies. This modular architecture, combined with CSV-based data persistence, produces a system that is lightweight, portable, and maintainable without specialised database administration skills.

The professional Tkinter-based GUI dashboard, featuring a live 30-fps camera feed, six dynamic status cards, a colour-coded activity log, and one-click access to all reporting functions, provides a usable and visually professional interface that non-technical administrative staff demonstrated they could operate effectively within five minutes of first use.

7.2 Challenges Encountered and Solutions

7.2.1 Threading Conflict Between OpenCV and Tkinter

The most significant technical challenge encountered during implementation was the conflict between OpenCV's requirement for imshow() to be called on the main thread and Tkinter's requirement for all widget updates to occur on the main GUI thread. Initial implementations placed the liveness detection loop in a background thread, which resulted in intermittent crashes and unresponsive windows on Windows 10.

The solution was the root.after() state machine pattern described in Section 4.4.3, which processes one OpenCV frame per callback invocation on the main thread without blocking the Tkinter event loop. This solution required a complete re-architecture of the camera management subsystem but produced a stable, crash-free implementation that has been validated across Windows 10 and Windows 11.

7.2.2 DeepFace Model Download on First Run

The DeepFace library downloads the VGG-Face model weights (approximately 500 MB)

from the internet on the first call to DeepFace.verify(). In environments with slow or restricted internet connectivity, this download could take several minutes and would block the GUI with no visible progress indicator. This was addressed by adding a first-run detection check in the GUI initialisation code that displays a dedicated progress message informing the user that the model is being downloaded, preventing confusion about whether the system is functioning correctly.

7.2.3 Matplotlib Threading Safety

Early implementations of the report generation functions called plt.show() to display charts, which caused application crashes when called from background threads on Windows. The solution was to replace plt.show() with the Agg backend approach described in Section 4.4.3: saving charts as PNG files and loading them into Tkinter Toplevel windows using PIL and ImageTk.

This approach is inherently thread-safe because it decouples chart rendering (which happens in the background thread) from chart display (which happens on the main thread via root.after() callbacks), and produces charts of consistent quality regardless of the operating environment.

7.3 Limitations of the Current System

While the system meets all defined objectives, several limitations of the current implementation are acknowledged. First, the system is designed for single-site, single-workstation deployment.

All data files reside on the local filesystem of the deployment machine, and there is no mechanism for synchronising attendance or access records across multiple computers or multiple physical access points. Organisations with multiple entrances or distributed sites would require the network-based architecture described in Section 7.4.

Second, the face recognition accuracy may degrade for individuals who undergo significant appearance changes after registration — such as growing a beard, acquiring glasses, or significant weight change — without re-registering.

This is a fundamental characteristic of template-based biometric systems rather than a specific limitation of this implementation; it can be mitigated operationally by establishing a periodic re-registration policy (for example, annual re-registration of all employees) or a triggered re-registration workflow when authentication failures increase for a previously reliable user.

Third, the anomaly detection module relies on rule-based classification with fixed thresholds (for example, RAPID_ACCESS_MINS = 5). These thresholds may require tuning for specific organisational contexts — an organisation that operates two shifts with a short changeover period might produce legitimate consecutive access events that trigger Rule 5. A future enhancement would make these thresholds configurable through the GUI administrator settings panel rather than requiring code modifications.

7.4 Future Work and Recommendations

7.4.1 Network-Based Multi-Door Deployment

The most impactful extension to the current system would be a network-based architecture supporting multiple access points. This would require replacing the local CSV data layer with a centralised database server (MySQL or PostgreSQL), implementing a lightweight REST API layer (using Python Flask or FastAPI) to expose data access endpoints, and deploying client instances at each access point that authenticate against the central server. The modular design of the current system ensures that only the data layer functions in each module would need to be modified to use API calls instead of direct file access.

7.4.2 Machine Learning-Based Anomaly Detection

The current rule-based anomaly detection engine could be enhanced with statistical and machine learning approaches that detect anomalies based on deviations from each employee's personal baseline behaviour rather than fixed absolute thresholds.

For example, an isolation forest model trained on each employee's historical access time distribution could flag access events that fall outside the employee's normal time-of-day range, even if the absolute hour does not fall within the fixed unusual hours window. This would reduce false positives for employees with legitimately unusual but consistent working patterns.

7.4.3 Physical Lock Hardware Integration

The current system operates as a software-only solution and does not control any physical door locking hardware. Integration with an electronic door strike or electromagnetic lock via a microcontroller relay (such as a Raspberry Pi GPIO output or an Arduino relay module) would transform the system into a complete physical access control solution. The module2_access.py module could be extended with a hardware control function that activates the relay for a configurable duration upon a GRANTED access decision.

7.4.4 Mobile Notification Integration

Security officers could benefit from real-time mobile notifications when security alerts are triggered. Integration with a messaging API such as Twilio SMS or WhatsApp Business API would enable the system to send an SMS or WhatsApp message containing the security alert details and the captured intruder photograph to a configured security officer contact number whenever a SECURITY_ALERT event is logged. This extension could be implemented as an optional notification plugin within module2_access.py.

7.5 Conclusion

This project has demonstrated that a comprehensive, multi-functional biometric access control system can be developed entirely from open-source Python libraries, achieving commercial-grade authentication accuracy and usability without any dedicated biometric hardware investment.

The system contributes to the field of applied biometric security engineering by presenting a validated, documented, and fully tested implementation of Challenge-Response Presentation Attack Detection using head movement instructions — a technique that is theoretically well-established but rarely documented in complete, deployable form in the academic FYP literature.

The modular five-module architecture, clean separation of concerns between the presentation, logic, and data layers, and complete test coverage of all 12 functional requirements provide a robust foundation for the future extensions identified in Section 7.4. The system is ready for immediate deployment in its current form at institutions with a single access point and a workforce of up to several hundred registered employees.

In conclusion, the Face Recognition Based Intelligent Access Control System with Attendance and Anomaly Analysis successfully fulfils all objectives set out in Chapter 1 and provides a meaningful contribution to the practical application of biometric security technology in institutional environments. The project demonstrates the viability of open-source deep learning tools for real-world security applications and provides a complete, reproducible implementation that can serve as a reference for future work in this domain.

REFERENCES

[1] S. I. Serengil and A. Ozpinar, "HyperExtended LightFace: A Speed-Accuracy Trade-off in Facial Attribute Analysis," 2021 International Conference on Engineering and Emerging Technologies (ICEET), Istanbul, Turkey, 2021, pp. 1-6.

[2] S. I. Serengil and A. Ozpinar, "LightFace: A Hybrid Deep Face Recognition Framework," 2020 Innovations in Intelligent Systems and Applications Conference (ASYU), Istanbul, Turkey, 2020, pp. 23-27.

[3] O. M. Parkhi, A. Vedaldi, and A. Zisserman, "Deep Face Recognition," in Proceedings of the British Machine Vision Conference (BMVC), 2015, pp. 41.1-41.12.

[4] OpenCV Development Team, "OpenCV: Open Source Computer Vision Library," Version 4.8. [Online]. Available: https://opencv.org/ [Accessed: June 2026].

[5] Python Software Foundation, "Python Programming Language," Version 3.10. [Online]. Available: https://www.python.org/ [Accessed: June 2026].

[6] The Pandas Development Team, "pandas: Powerful Python Data Structures for Data Analysis," Version 2.0. [Online]. Available: https://pandas.pydata.org/ [Accessed: June 2026].

[7] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, May 2007.

[8] A. K. Jain, A. Ross, and S. Prabhakar, "An Introduction to Biometric Recognition," IEEE Transactions on Circuits and Systems for Video Technology, vol. 14, no. 1, pp. 4-20, Jan. 2004.

[9] ISO/IEC 30107-3:2023, "Information Technology - Biometric Presentation Attack Detection - Part 3: Testing and Reporting," International Organization for Standardization, Geneva, Switzerland, 2023.

[10] Y. Li, K. Xu, and J. Yan, "Face Anti-Spoofing Using Deep Learning: A Comprehensive Survey," IEEE Access, vol. 9, pp. 129526-129544, 2021.

[11] G. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, "Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments," University of Massachusetts, Amherst, Tech. Rep. 07-49, Oct. 2007.

[12] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. Cham, Switzerland: Springer, 2022.

[13] I. Sommerville, Software Engineering, 10th ed. Harlow, UK: Pearson Education, 2016.